

几何视域下板凳龙在盘龙全过程的姿态研究

摘要

“板凳龙”，是一种传统民俗文化活动。本文我们将盘龙全过程抽象成数学表示，并在平面几何视域下进行姿态参数的解算，模拟了舞龙过程的运动轨迹和空间需求，这不仅有助于提高板凳龙表演的观赏性，还可以为其他类似的民俗活动提供参考。

对于问题 1，为了有效利用阿基米德螺线的性质，我们先将盘入问题逆向转化为由原点螺旋而出的情形，并利用 `ode45` 函数求解出了关于龙头点极角时间函数微分方程的数值解，为了对各时刻各把手点位置与速度迭代求解，我们构造递归圆并结合几何关系，求解出了前后把手点位置递推关系并将极坐标转为笛卡尔坐标，然后利用速度分解得出了前后把手速度递推关系，完整结果见 `result1.xlsx`。

对于问题 2，为了确定使板凳间不碰撞的盘入终止时刻，我们划分了 3 种板凳间位置关系，并计算向量内积来界定板凳间碰撞与否，最后求得终止时刻约为 $412.474054s$ ，此时龙头板凳角与龙头后第 8 条龙身板凳边相撞并进行可视化，此刻的各把手位置与速度完整结果见 `result2.xlsx`。

对于问题 3，为了保证龙头前把手能沿着螺线盘入调头空间的边界，并在此之前不会有板凳发生碰撞，我们使用逐步递减搜索步长的搜索方法来求解此优化模型，求解出的最小螺距为 $d_m = 0.450340m$ ，与调头区域边界交点的坐标为 $(3.848418, 4.852865)$ ，并与更小螺距 $d = 0.44m$ 的相撞情形进行了可视化比较，佐证了此最小螺距的合理性。

对于问题 4，我们首先以最小化调头空间中路径总长度为优化目标，得出该路径总长度最小值为 $13.714630m$ ，对应有有效圆半径 $R = 4.5m$ ，我们还使用最小化调头曲线中两段圆弧总长为优化函数，以不会相撞、卡死为约束条件，得出两段弧总长最小值为 $13.059333m$ 时刚好不卡死，其对应于 $R = 4.29m$ ，并将两种情形可视化，由于调头路径与盘入盘出轨迹有很大不同，我们先界定了 6 个临界点将调头路径划分成了 7 个路径段并对每段中板凳姿态进行了可视化，随后逐段利用几何关系推导出了位置递推关系和速度递推关系，分 $R = 4.5m$ 和 $R = 4.29m$ 两种路径迭代求解出了调头前后全过程各时刻所有把手点的位置与速度， $R = 4.5m$ 时的完整结果见 `result4.xlsx`。

对于问题 5，为计算龙头前把手的最大行进速度，我们首先求出了所有把手点最大速度趋势上在特定时刻出现尖峰并进行可视化，依此我们建立基本优化模型求解出了当 $R = 4.5m$ 时，龙头最大行进速度为 $1.232m/s$ ，所有把手点所有时刻的最大速度为 $1.9998m/s$ ，当 $R = 4.29m$ 时，龙头最大行进速度为 $0.6313m/s$ ，所有把手点所有时刻的最大速度为 $1.9908m/s$ 。

在模型检验部分，我们依据等距螺线的弧长公式度量了问题 1 的数值解误差处于 $1e^{-6}$ 的数量级，具有极高的精度，同时根据速度大小方向与位置的关系图，对问题 5 中尖峰现象可视化检验，保证了模型的合理性，并探究了模型的评价与推广。

关键字： 板凳龙 阿基米德等距螺线 平面几何 迭代递推关系

一、问题重述

1.1 问题背景

随着中国经济逐渐走向世界舞台中央，在物质生活提升的同时我们也逐渐倡导精神生活的丰富以及文化的熏陶，这使得中国传统文化再次引起了普罗大众的关注。“板凳龙”，又称“盘龙”，作为一种传统民俗文化活动，长久以来极大地丰富了浙闽地区人们的文化生活，乃至对其理性精神和商业精神有着极其重大的影响[1]。在这项活动中，板凳首尾相连形成蜿蜒曲折的板凳龙。盘龙时，龙头引领方向，龙身和龙尾则灵活地盘旋，形成一个动态的圆盘，如图1所示。通常情况下，舞龙队伍能够自如地盘入和盘出是舞龙取得成功的关键因素。盘龙所需的面积越小，行进速度越快，观赏性就越高。



图1 板凳龙示意图

1.2 问题提出

“板凳龙”这项民俗活动是我国传统文化中极其宝贵的财富，需要拥有精湛技艺的传承人的代代传承，但在该民俗活动中，如何在有限的空间内实现高效的盘龙表演在数学建模方面仍有迹可循，我们力图通过建立数学模型来模拟板凳龙的运动轨迹和空间需求，这不仅有助于提高板凳龙表演的观赏性，还可以为其他类似的民俗活动提供参考。同时，我们在这一过程中也可以更好地理解 and 提升这一传统活动的表现效果。

我们需要建立模型解决以下问题：

问题 1 舞龙队沿螺距为 55cm 的等距螺线顺时针盘入，各把手中心均位于螺线上。龙头前把手的行进速度始终保持 1m/s。已知初始位置，计算从初始时刻到 300s 为止，每秒整个舞龙队的位置和速度。

问题 2 舞龙队沿问题 1 设定的螺线盘入，确定舞龙队盘入的终止时刻，使得到终止时刻时，板凳之间不发生碰撞，并计算此时舞龙队的位置和速度。

问题 3 从盘入到盘出，舞龙队将由顺时针盘入调头切换为逆时针盘出，这需要一定的调头空间。假定调头空间是以螺线中心为圆心、直径为 9m 的圆形区域，确定最小螺距，使得龙头前把手能够沿着相应的螺线盘入到调头空间的边界。

问题 4 假定螺距为 1.7m，盘出与盘入螺线关于螺线中心呈中心对称，舞龙队在问题 3 设定的调头空间内保持 1m/s 的行进速度调头，路径是两段圆弧组成的 S 曲线，前一段圆弧的半径是后一段的 2 倍。保持圆弧之间以及与螺线之间均相切，调整圆弧使得调头曲线变短。以调头开始时间作为零时刻，计算每秒舞龙队的位置和速度。

问题 5 舞龙队沿问题 4 设定的路径行进，龙头行进速度保持不变，计算龙头的最大行进速度，使得舞龙队各把手的速度均不超过 2m/s。

二、模型假设

为便于模型的建立与求解，现作如下假设：

1. 舞龙队在盘入盘出时以及进入 A 点前，各把手中心形成的轨迹看作多圈等距螺线。

舞龙队盘龙过程形成的轨迹类似圆盘状，可看作等距螺线，在舞龙队开始盘龙过程之前一般也会首先调整姿态呈近似等距螺线状，以方便后续盘龙，所以我们可以假设舞龙队在盘入盘出时以及进入 A 点前均绕等距螺线圈轨迹，这样可以将现实问题转化为数学表示，将复杂问题进行抽象简化以方便求解，各把手中心在等距螺线上这个约束条件也在大大降低了问题求解的复杂度的同时保证合理性。

2. 盘龙时龙头前把手保持匀速，整个队伍位置速度用各个前后把手中心的指标表示。

盘龙时各板凳的位置与速度与前后板凳的位置与速度都有关系，题目假设龙头前把手保持匀速可以约束其他把手速度的连贯性，题目也建议使用各板凳前后把手中心的位置与速度来表示整个队伍某个时刻的状态，并填到附件表格中，这大大简化了问题计算复杂度，也更符合实际情况。

3. 在舞龙队盘入、调头、盘出的整个过程中，板凳均为刚性的，且不考虑外界干扰。

由于环境剧变、体力不支等均为小概率事件，且板凳形变、摩擦老化等均会造成不确定性增加，会急剧提升分析复杂度，我们假设这些并不存在，这样在降低繁杂性的同时也利于模型的建立。

三、符号说明

符号	具体意义	单位
d	阿基米德螺线螺距	m
$\theta(t)$	螺线从原点开始 t 时刻经过的角度	rad
$r(t)$	t 时刻螺线点到原点的距离	m
v_1	t 时刻垂直于 r 路径的速度	m/s
v_2	t 时刻沿 r 路径方向的速度	m/s
v_0	龙头前把手中心点的行进速度	m/s
v_i	第 i 个把手中心点的速度	m/s
(x_i, y_i)	第 i 个点或 P 点的直角坐标	(m, m)
t_f	从原点 O 逆向螺旋到起始点 A 所需的时间	s
θ_{ij}	从第 i 个把手点到第 j 个经过的角度变化值	rad
L_{ij}	板凳前后两个把手中心点之间的刚体距离	cm
H_h	龙头板凳完整长度	cm
H_e	龙身、龙尾板凳完整长度	cm
θ_{dij}	从第 i 个到第 j 个把手的板凳方向相对于初始极轴角度	rad
θ_v	第 i 个把手点速度方向相对初始极轴转过的角度	rad
θ_{di}	从第 i 个把手点与板凳方向的夹角	rad
β	目标点与矩形板凳两个相邻的边构成的三角形的邻近矩形边的内角	rad
R	调头区域净有效圆的半径	m
θ_{iM}	第 i 个点在圆心为 M 的圆弧上转过的角度值	rad
θ_{diM}	第 i 个点在圆心为 M 的圆弧上速度方向与板凳方向的夹角	rad

注：由于符号琐碎，这里只展示部分符号，其他说明见正文。

四、问题分析

4.1 问题总分析

总览本题，可以看出本赛题涉及舞龙队在盘龙过程中构成的阿基米德等距螺线相关问题，该等距螺线由各个板凳的前后把手中心点来约束，我们需要建模求解盘龙过程以及调头过程的每个时刻的各把手点的位置与速度，同时也需要保证板凳之间不发生碰撞，在调头空间的掉头问题上，即盘入盘出的转变，还涉及到螺距和调头路径的优化问题，以及探讨龙头行进速度变化对舞龙队各把手速度的影响。

4.2 问题一分析

问题 1 假定舞龙队各板凳把手中心沿等距螺线顺时针盘入，已知初始位置和龙头前把手的行进速度，计算每秒整个舞龙队的位置和速度。要求解本问题，我们可以先将盘入问题逆向转化为由原点螺旋而出的情形，然后根据阿基米德等距螺线的基本性质来推导出极坐标系下螺旋线上点的坐标随时间变化关系，以此来计算龙头前把手的位置，并可通过递归圆来求解其他把手中心点的位置坐标，依据此坐标并利用速度分解来迭代计算出其他把手中心点的速度，从而得解。

4.3 问题二分析

问题 2 中舞龙队轨迹与问题 1 相同，要求确定使得板凳之间不发生碰撞的盘入终止时刻，并计算此刻舞龙队的位置和速度。求解本问题，我们需要界定发生碰撞的临界状态，由此来分析板凳之间的多种位置关系，从而解算出恰好碰撞的时刻，此刻的位置与速度求解与问题 1 类似。

4.4 问题三分析

问题 3 开始考虑从顺时针盘入到逆时针盘出的转变，即舞龙队到达调头空间之前轨迹受螺距大小的制约情况，并求出最小螺距。本问题假定调头空间是圆形区域，除了考虑使得舞龙队沿螺线进入调头区域之外，我们还需要保证板凳之间不相撞，这是一个相对标准的优化问题，轨迹受螺距大小制约关系相对单调，所以我们可以逐渐减小螺距来进行迭代求解。

4.5 问题四分析

问题 4 固定螺距大小，约束螺线关系和调头路径，综合考虑调头空间内外路径情况，要求探究调整圆弧对调头路径的影响，并计算调头前后各时刻各把手的位置和速度。要解答本问题，我们需要在不发生碰撞、不造成龙头被龙身卡死的前提下，根据螺线关系

和调头路径的牵制关系来求解最短的调头曲线，由于调头路径与等距螺线的轨迹不同，所以我们需要对路径进行分段，逐个攻破，求出各自的位置速度递推关系，从而能够迭代求解。

4.6 问题五分析

问题 5 路径与问题 4 相同，要求在保证各时刻各把手点的速度均不超过 2m/s 的前提下，计算龙头前把手的最大行进速度。我们可以先求出所有把手点最大的速度随时刻变化的趋势，这本质上是一个单变量控制的优化问题，然后使用最为基础的固定步长逐步增加龙头行进速度，从而实现优化问题的迭代求解。

五、模型建立与求解

5.1 问题一求解

5.1.1 由内向外 $\theta(t)$ 及 $r(t)$ 的求解

由于我们将舞龙队盘龙过程中的轨迹视作阿基米德等距螺线，所以在这里我们首先进行等距螺线公式的相关推导，我们在这里假设一个点从原点出发，沿逆时针方向做匀角速度运动，产生阿基米德等距螺线，如图 2 所示，但在问题中描述的是由外向内盘入，如图 3 所示，这一点我们将在 5.1.3 中进行进一步的解释。

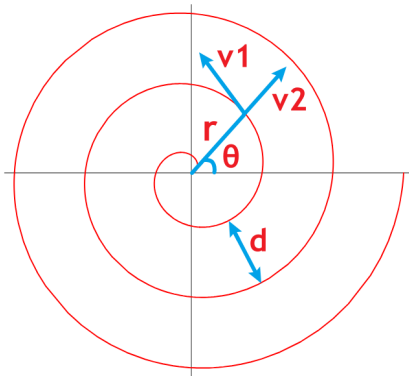


图 2 阿基米德等距螺线示意图

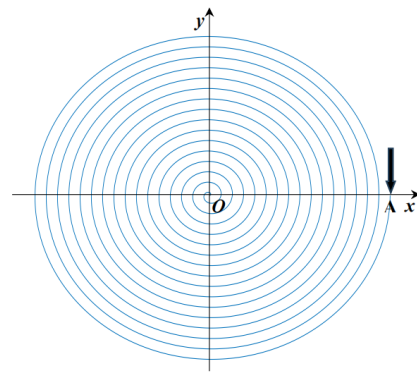


图 3 盘龙时龙头起始点示意图

在图 2 中， d 代表等距螺线的螺距，在本问中 $d = 0.55m$ ， θ 表示螺线从原点开始 t 时刻经过的角度， r 表示 t 时刻螺线点到原点的距离， v_1 表示 t 时刻垂直于 r 路径的速度， v_2 表示 t 时刻沿 r 路径方向的速度，我们可以得到：

$$\begin{cases} r = v_2 t \\ \theta = \omega t \end{cases} \quad (1)$$

当螺线点经过 Δt , 角度转过 $\Delta\theta$, 到原点的路径距离增加 Δr , 而当变化角度 $\Delta\theta = 2\pi$ 时, 恰好有 $\Delta r = d$, 从而得到:

$$\begin{cases} \Delta\theta = 2\pi = \omega t \\ \Delta r = d = v_2 t \end{cases} \quad (2)$$

联立 (1) 与 (2) 可得到:

$$\begin{cases} v_1 = \omega r = \omega v_2 t = v_2 \theta \\ v_2 = \frac{\omega d}{2\pi} \end{cases} \quad (3)$$

题目中给出龙头前把手的行进速度始终保持 $v_0 = 1m/s$, 沿 r 方向切向速度和法向速度的叠加即为该行进速度, 所以我们有:

$$\begin{cases} v_1^2 + v_2^2 = v_0^2 = 1 \\ v_1 = v_2 \theta \end{cases} \quad (4)$$

由此式我们可以求解出 v_1 和 v_2 :

$$\begin{cases} v_1 = \frac{\theta}{\sqrt{\theta^2 + 1}} \\ v_2 = \frac{1}{\sqrt{\theta^2 + 1}} \end{cases} \quad (5)$$

同时我们结合等距阿基米德螺线的基本性质和 (1) 可以得到:

$$\begin{cases} r = \frac{d\theta}{2\pi} \\ r = \int \frac{1}{\sqrt{\theta^2 + 1}} dt \end{cases} \quad (6)$$

从而我们只需求解 (7) 即可得出 θ 与 t 的关系:

$$\frac{d}{2\pi} \theta'(t) = \frac{1}{\sqrt{\theta^2 + 1}} \quad (7)$$

然后我们带入 $d = 0.55m$ 对 (7) 进行求解, 可以得出:

$$\frac{\ln(\sqrt{\theta^2 + 1} + \theta)}{2} + \frac{\theta\sqrt{\theta^2 + 1}}{2} = \frac{40\pi t}{11} + C \quad (8)$$

但是我们无法由 (8) 得出 $\theta(t)$ 的解析解, 所以我们使用 *matlab* 的 *ode45* 函数求出 (7) 的数值解, 并使用 *odeset* 函数来使数值解的精度达到 $1e^{-10}$, 然后将每个时刻 t 的值代入, 即可得到对应的 θ 值, 利用 (6) 的第一个式子我们还可以得出对应的 r 值, 同时我们利用此式也可实现 θ 和 r 的相互转换。

5.1.2 极坐标系和笛卡尔坐标系的转换

在得到 θ 和 r 的值后，我们就可利用极坐标和笛卡尔坐标系的转换关系来得到每个时刻每个把手的 (x, y) 坐标，在后续部分中，我们计算出每个把手点的极坐标后，均会按照下式来进行极坐标向直角坐标的转换 [2]：

$$\begin{cases} x = r \cos \theta \\ y = r \sin \theta \end{cases} \quad (9)$$

5.1.3 盘入过程的逆向转化

由于在上述过程中我们是从原点由内向外螺旋的，这很方便于求解，但题目中要求的是从起始点 A 点位置由外向内盘入，这实际上只需要将求解时间顺序逆转过来即可，由于龙身和龙尾受龙头的牵连，所以在这里我们只考虑龙头的前把手对应的点，这是因为该点符合我们在5.1.1部分描述的简化求解过程，其余点的位置会根据龙头前把手点的位置而固定，具体求解会在5.1.5部分进行阐述。在逆向转化的过程中，我们首先计算了该点从原点由内向外螺旋到图3所示的 A 点所需要的时间：

$$\begin{cases} r(t_f) = 16 \times 0.55 = 8.8m \\ t_f \approx 442.5904725s > t_1 = 300s \end{cases} \quad (10)$$

由 (10) 我们可以看到，从原点到 A 点的时间大于问题1中要求的计算位置与速度的时间段，也就是说，我们可以逆向求解出各个时刻的龙头前把手点的位置，相当于板凳龙从原点开始往外倒出的过程，一些时刻可以进行正向逆向的相互对应，具体来说，逆向过程的 t_f 时刻对应于正向的 $0s$ 时刻，逆向过程的 $(t_f - 300s)$ 时刻对应于正向的 $300s$ 时刻，在程序代码中我们对时刻进行了转换，使逆向求解过程的时刻对应于盘入过程的 $0s \sim 300s$ 的各个时刻。

5.1.4 各时刻整个舞龙队所有把手中心点位置的求解

除5.1.3部分对于龙头前把手点的位置求解外，其他把手点的位置可以通过递归圆来求解，如图4所示。

为与5.1.1部分相一致，这里我们仍以由内而外的螺旋过程进行阐释，逆向转化过程参见5.1.3部分，在图4中，我们展示了从龙头到龙身龙尾向外螺旋的板凳的连接关系及位置示意，其中 r_i 表示从龙头板凳开始 (包括龙头板凳) 向外螺旋过程的第 i 个板凳前把手点 P 到原点 O 的距离，同理， r_j 与 r_m 分别表示紧随其后的第 j 个和第 m 个， θ_{ij} 表示从 P 点到 Q 点经过的角度变化值， L_{ij} 表示从龙头板凳开始 (包括龙头板凳) 向外螺旋过程的第 i 个板凳的前后两个孔把手点之间的刚体距离，龙头板凳与其他板凳的

在求出 r_j 后，我们再依据该距离画圆逐一递归迭代出每个把手中心的位置，然后再按照5.1.2部分进行坐标转换，以及5.1.3部分进行每个时刻的逆向转化，从而得到从初始时刻到 300s 为止，每秒龙头、龙身和龙尾各前把手及龙尾后把手中心的位置，完整结果见附件 *result1.xlsx*，这里只展示部分结果，如表 1 所示，所有把手中心点在 300s 时的位置可视化图见图 5，可以看出，在 300s 时，整个板凳龙仍未将所有板凳行进至完全进入如图 3 所示的 16 圈等距螺线内。

表 1 舞龙队部分时刻部分把手中心点的位置坐标

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 x(m)	8.800000	5.799363	-4.084707	-2.963804	2.594690	4.420177
龙头 y(m)	0.000000	-5.770941	-6.304599	6.094688	-5.356651	2.320622
第 1 节龙身 x(m)	8.363757	7.456850	-1.445261	-5.237258	4.821352	2.459302
第 1 节龙身 y(m)	2.826749	-3.440204	-7.405926	4.359463	-3.561777	4.402584
第 51 节龙身 x(m)	-9.518764	-8.686380	-5.543314	2.890256	5.980129	-6.301365
第 51 节龙身 y (m)	1.340923	2.539901	6.377806	7.249371	-3.827578	0.465614
第 101 节龙身 x (m)	2.914191	5.687293	5.362116	1.899005	-4.917202	-6.237839
第 101 节龙身 y (m)	-9.918252	-8.001260	-7.557515	-8.471569	-6.380008	3.935827
第 151 节龙身 x (m)	10.861692	6.682146	2.388548	1.004940	2.965175	7.040628
第 151 节龙身 y (m)	1.828967	8.134682	9.727465	9.424776	8.399794	4.393197
第 201 节龙身 x (m)	4.554904	-6.619839	-10.627240	-9.287632	-7.457015	-7.458540
第 201 节龙身 y (m)	10.725204	9.025444	1.359633	-4.246870	-6.180893	-5.263561
龙尾（后） x (m)	-5.305251	7.364724	10.974333	7.383743	3.240848	1.784822
龙尾（后） y (m)	-10.676682	-8.797854	0.843689	7.492523	9.469408	9.301207

5.1.5 各时刻整个舞龙队所有把手中心点速度的求解

在求解各个把手中心点的位置后，我们还需要求解每个把手中心点的速度，而我们已知龙头前把手中心点的速度为 1m/s，其他把手中心点的速度与其并不相同，前后把手的速度关系如图 6 所示。

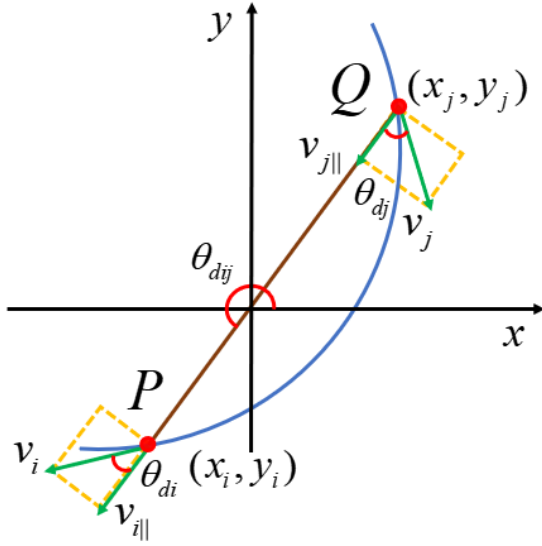


图 6 前后把手速度关系示意图

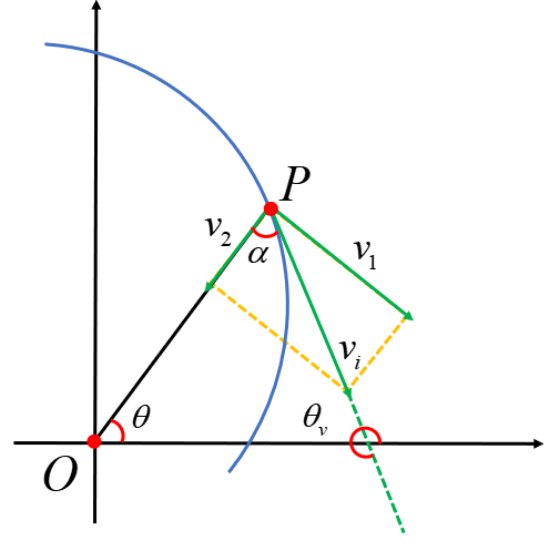


图 7 把手中心点速度相对初始极轴角度

结合刚体力学相关知识 [3]，我们可以知道前后把手中心点的速度沿板凳方向的分解速度是一样的，以此我们可以串联起各个把手中心点的速度，进行迭代求解，如图 6 所示， θ_{dij} 表示第 i 个把手中心点和第 j 个把手中心点之间板凳方向相对于初始极轴的角度，前后把手速度关系如下：

$$\begin{cases} v_{i||} = v_i \cos \theta_{di} \\ v_{j||} = v_j \cos \theta_{dj} \\ v_{i||} = v_{j||} \end{cases} \Rightarrow v_i \cos \theta_{di} = v_j \cos \theta_{dj} \quad (12)$$

我们知道， $v_0 = 1m/s$ ，可用龙头前把手速度进行后续速度的迭代，只需要知道每两个把手中心点速度相对于板凳方向的 θ_{di} 和 θ_{dj} ，而这两个角度实质上求解方法是一样的，这是因为这些相对角度本质上是各个把手中心点速度相对于板凳方向的夹角，而我们在部分已经求得了各个时刻各把手中心点的位置坐标，所以我们可求出如图 6 所示的 θ_{dij} ，这需要在笛卡尔坐标系下求解：

$$\tan \theta_{dij} = \frac{y_j - y_i}{x_j - x_i} \Rightarrow \theta_{dij} = \arctan \frac{y_j - y_i}{x_j - x_i} \pm n\pi, n \in N^* \quad (13)$$

然后我们需要计算第 i 个把手中心点速度方向相对于初始极轴转过的角度，如图 7 所示，即为对 θ_v 的求解，由 (4) 第二个式子我们得出 α 角度值，结合之前求出的 θ 角度值，利用三角形内外角关系，即可求出 θ_v ，具体如下：

$$\begin{cases} v_1 = v_2 \theta \\ \theta_v = \alpha + \theta + \pi \end{cases} \Rightarrow \begin{cases} \alpha = \arctan \theta \\ \theta_v = \arctan \theta + \theta + \pi \end{cases} \quad (14)$$

注意，在这里我们求解出的 θ_v 最后加了一个 π ，这是只有盘入时才有的，后续问题中盘出时则没有这个 π ，此外，上述对于 θ_v 求解的过程是基于直角坐标系第一象限的，而我们实际上推导了四个象限的求解过程，得出的结果是一样的，在此不再赘述。

在求解出 θ_{dij} 和 θ_v 后，我们将两者相减即可得到图 6 中的 θ_{di} ，如下：

$$\theta_{di} = |\theta_v - \theta_{dij}| \in (0, \frac{\pi}{2}) \quad (15)$$

注意，由于求解出来的角度差会 $\mp$ 多个 π ，使得结果变得繁琐，而各个把手中心点的速度方向相对板凳方向的角度差必然是一个锐角，所以我们只需要保留小于 $\frac{\pi}{2}$ 的角度差，去除其他不合理的角度差，同理，可以求出 θ_{dj} ，我们由已知 v_i ，从而解算出 v_j ，从龙头前把手中心点逐步迭代到龙尾后把手中心点，即可求解出所有把手中心点的速度，具体逻辑如下：

$$\begin{cases} \theta_v = \arctan \theta + \theta + \pi \\ \theta_{dij} = \arctan \frac{y_j - y_i}{x_j - x_i} \pm n\pi \\ \theta_{di} = |\theta_v - \theta_{dij}| \\ v_i \cos \theta_{di} = v_j \cos \theta_{dj} \end{cases} \Rightarrow \begin{cases} \theta_{di} = |\arctan \theta + \theta - \arctan \frac{y_j - y_i}{x_j - x_i}| \in (0, \frac{\pi}{2}) \\ v_j = \frac{v_i \cos \theta_{di}}{\cos \theta_{dj}} \end{cases} \quad (16)$$

对于各把手中心点的速度，完整结果见附件 **result1.xlsx**，这里只展示部分结果，如表 2 所示。

表 2 舞龙队部分时刻部分把手中心点的速度大小

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999971	0.999961	0.999945	0.999917	0.999859	0.999709
第 51 节龙身 (m/s)	0.999742	0.999662	0.999538	0.999331	0.998941	0.998065
第 101 节龙身 (m/s)	0.999575	0.999453	0.999269	0.998971	0.998435	0.997302
第 151 节龙身 (m/s)	0.999448	0.999299	0.999078	0.998727	0.998115	0.996861
第 201 节龙身 (m/s)	0.999348	0.999180	0.998935	0.998551	0.997894	0.996574
龙尾 (后) (m/s)	0.999311	0.999136	0.998883	0.998489	0.997816	0.996478

5.2 问题二求解

本问题延续了问题 1 的盘入螺旋线轨迹，要求计算出舞龙队盘入的终止时刻，使得板凳之间不发生碰撞，所以我们需要计算出板凳之间恰好相撞的时刻，对于板凳相撞的情况，由于在本题中，等距螺线看作两孔之间的线段拟合的曲线，而随着盘龙过程越往里曲率越大，所以在头部的板凳更容易以角撞边的形式与龙身相撞，可以得出板凳最先相撞的位置一定是板凳角点与板凳边相撞，具体可分为如图 8 所示的三种情况。

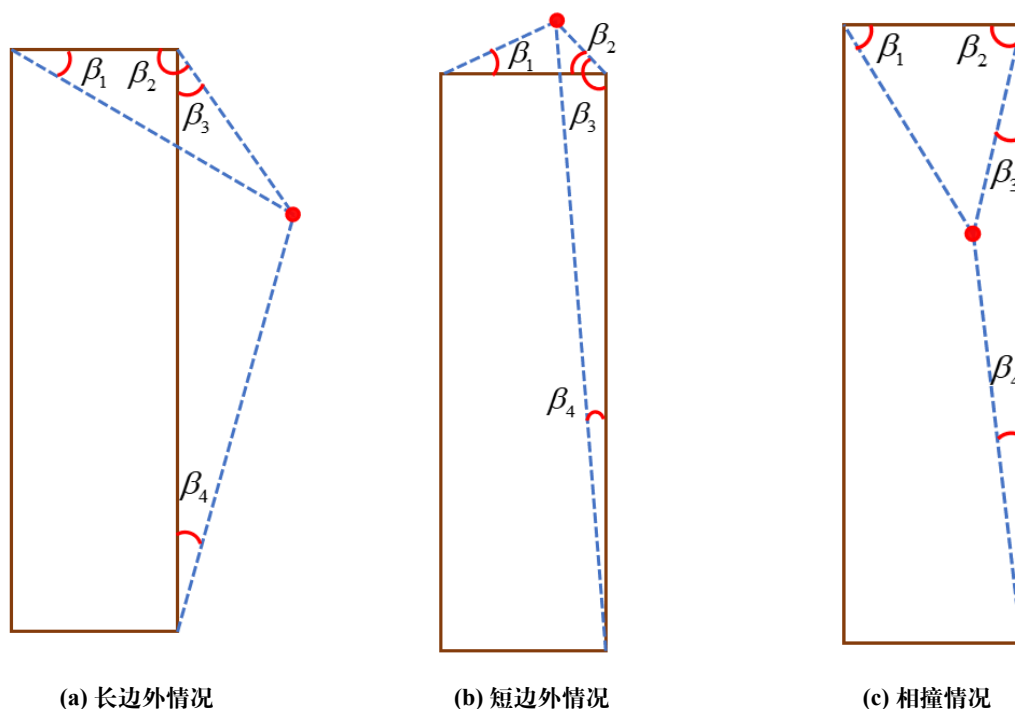


图 8 板凳是否相撞三种情况示意图

由于我们在问题 1 中能够计算各把手点的位置与速度，我们也已知龙头板凳与龙身龙尾板凳的尺寸，所以我们可以计算出每个板凳各个角对应的坐标，由于这只涉及到基础的平面坐标系几何计算，我们在这里不再赘述由各把手点坐标计算板凳角坐标的过程，而是将重点放在对于碰撞的判断上，如图 8 所示，我们展示了判断板凳是否相撞的三种情况，在图 8a 中，目标点在板凳之外，在 $\beta_1\beta_2\beta_3\beta_4$ 中 β_2 是钝角，在图 8b 中，目标点在板凳之外， β_3 是钝角，在图 8c 中，板凳发生相撞， $\beta_1\beta_2\beta_3\beta_4$ 均为锐角，也就是说，未发生相撞时，目标点与目标板凳的任意三个板凳角点连线构成的三角形，即目标点与矩形板凳两个相邻的边构成的三角形的邻近矩形边的内角中必然有钝角，一旦发生相撞，越过临界情况，则对应的四个角全变成锐角，要判断是否存在钝角，我们很自然地可以想到使用向量内积来判断正负来区分锐角钝角，而在求解问题 1 的过程中，所有板凳的四个角的坐标均可求出，坐标相减即可得到方向向量，在迭代过程中向量点乘全变为正值即说明已经发生相撞，由于对所有板凳角点坐标两两相减再两两之间做点乘会提高计算复杂度，而在舞龙队盘入的整个过程中，距离太远的板凳不会发生碰撞，我

们只需要考虑相互距离较近的板凳角点，由于涉及到两个维度的距离度量，我们在这里使用曼哈顿距离来进行碰撞计算的取舍，我们取曼哈顿距离阈值为 5m，只有两个板凳角点之间距离小于这个阈值才有碰撞的可能性，本过程的核心为 (17) 的向量点乘判断，具体的处理过程及细节详见附件代码。

$$\begin{cases} P(x_i, y_i) \\ Q(x_j, y_j) \\ M(x_m, y_m) \end{cases} \Rightarrow \begin{cases} \vec{PQ} = \vec{OQ} - \vec{OP} \\ \vec{PM} = \vec{OM} - \vec{OP} \end{cases} \Rightarrow \cos \beta = \frac{\vec{PQ} \cdot \vec{PM}}{|\vec{PQ}| \times |\vec{PM}|} \begin{cases} < 0 & miss, \\ > 0 & collision. \end{cases} \quad (17)$$

经逆向过程转化后，我们解出相撞时刻即舞龙队盘入的终止时刻约为 412.474054s，所有板凳在终止碰撞时刻的位置姿态可视化结果见图 9，我们可以看到，此时整条板凳龙已经完全进入了图 3 所示的多圈等距螺线，龙头板凳与龙头后面第 8 条龙身板凳相撞，其余板凳均未相撞，且相撞情况确实为板凳角撞板凳边，符合我们之前的推理，对于在终止时刻舞龙队的位置和速度的完整结果见附件 *result2.xlsx*，这里只展示部分结果，如表 3 所示。



图 9 终止时刻各板凳的位置姿态图

表 3 舞龙队终止时刻部分把手中心点的位置坐标和速度大小

	横坐标 x (m)	纵坐标 y (m)	速度 (m/s)
龙头	1.209931	1.942784	1.000000
第 1 节龙身	-1.643792	1.753399	0.991551
第 51 节龙身	1.281201	4.326588	0.976858
第 101 节龙身	-0.536246	-5.880138	0.974550
第 151 节龙身	0.968840	-6.957479	0.973608
第 201 节龙身	-7.893161	-1.230763	0.973096
龙尾（后）	0.956217	8.322736	0.972938

5.3 问题三求解

在本问题中螺距可变，需要确定最小螺距使得能够沿螺线盘入调头空间，注意这里并非与边界相切，严格来说是相交关系，从而进入调头空间，而螺距越小越容易实现，理论上可以无限小，但是螺距越小板凳之间越容易发生碰撞，所以我们的目标转变成确保沿螺线盘入调头空间，并在这之前不发生板凳间相撞，即在沿螺线盘入调头空间之前不会被迫终止，于是在这里我们可以沿用问题 2 的相撞临界判断方法，可以得到如下优化问题：

$$\min d \Rightarrow d_m \quad \text{s.t.} \quad \begin{cases} \exists t_c, r(t_c) = 4.5m, \\ \text{before } t_c, \forall M \in \mathbb{R}^2, \exists \cos \beta < 0. \end{cases} \quad (18)$$

其中， t_c 表示沿螺线盘入调头区域的时刻， $\mathbb{R}^2$ 表示板凳角点构成的点集， β 表示 5.2 部分提及的目标点与矩形板凳两个相邻的边构成的三角形的邻近矩形边的内角，由于此优化问题较为标准，没有较高的计算复杂度，参数变化趋势也比较单调，所以我们在这里不需要使用较为复杂的优化方法，最为基础的固定步长逐步递减参数优化便可解决问题，于是我们逐步减小螺距，直至恰好发生相撞之前，并在此过程中达到能够经过与原点距离恰好 4.5m 的螺线上的点位的要求。

最后我们求解出的最小螺距为 $d_m = 0.450340m$ ，与调头区域边界交点的坐标为 (3.848418, 4.852865)，最小螺距时对应的各板凳位置姿态如图 10a 所示，我们可以看到，板凳龙头前把手恰好能够沿着相应的螺线盘入到调头空间的边界，稍微小于最小螺距时对应的各板凳位置姿态如图 10b 所示，这里我们使用的是 $d = 0.44m$ ，我们可以看到，已经有板凳相撞的点了，在到达与调头区域交点的位置之前，龙头板凳后端角与龙头后

面第 19 条龙身板凳边发生碰撞，这进一步佐证了比最小螺距更小的螺距不再满足题目要求，我们求出的 $0.450340m$ 的最小螺距是合理的。

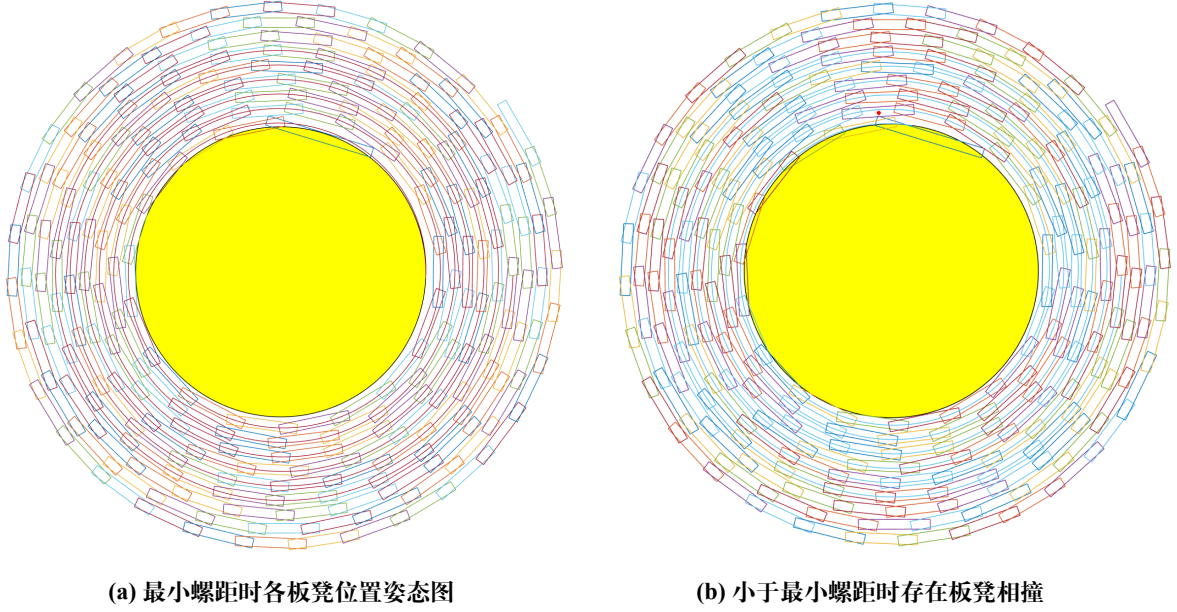


图 10 不同螺距时各板凳位置姿态图

5.4 问题四求解

5.4.1 调头路径相关参数的求解

本问题中，螺距大小固定为 $d_s = 1.7m$ ，盘出螺线与盘入螺线关于螺线中心呈中心对称，调头空间与问题 3 相同，调头路径是由两段圆弧相切连接而成的 S 形曲线，前一段圆弧的半径是后一段的 2 倍，它与盘入、盘出螺线均相切，根据这些约束条件，我们可以尝试计算调头路径的相关参数，并尝试解析这个调头路径，如图 11 所示。

根据阿基米德等距螺线的基本性质，我们可以求解 S 点的极坐标参数：

$$\begin{cases} r(t_s) = R = 4.5m \\ r(t_s) = \frac{d_s}{2\pi}\theta(t_s) \end{cases} \Rightarrow \theta(t_s) = \frac{90\pi}{17} \quad (19)$$

其中 t_s 表示龙头前把手中心点到达调头区域边界的时刻， R 表示调头区域的半径大小，在图 11 中，根据几何关系我们可以得到：

$$\begin{cases} \angle OSx = \theta(t_s) - n\pi \in (0, \frac{\pi}{2}) \\ \theta_v(t_s) = \arctan \theta(t_s) + \theta(t_s) + \pi \\ \angle MSx = \theta_v(t_s) - \frac{\pi}{2} \\ \theta_s = \angle OSx - \angle MSx \end{cases} \quad (20)$$

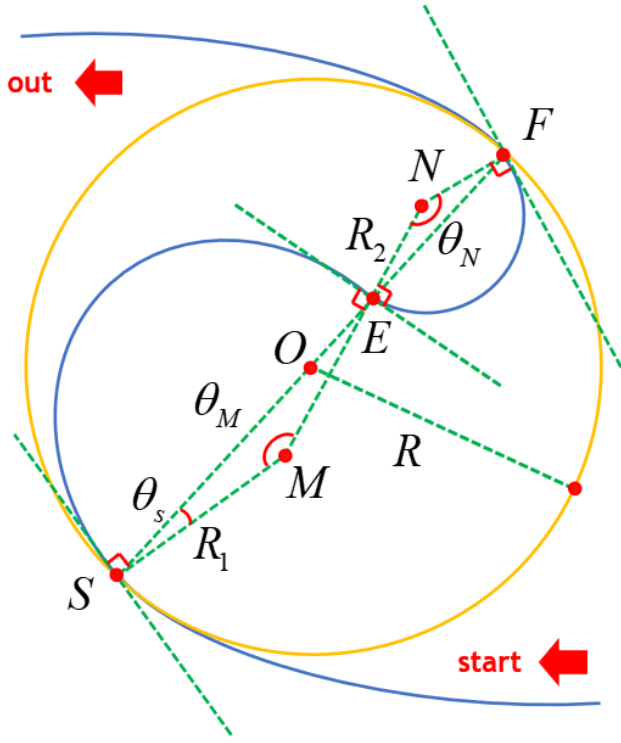


图 11 调头路径示意图

其中, $\theta_v(t_s)$ 表示在 t_s 时刻, 螺线点速度方向相对于初始极轴正方向转过的角度变化值, θ_s 表示两段圆弧圆心角三角形的底角, 根据如图 11 所示的几何关系, 我们可以得出这四个角的大小均相同, 从而我们可以利用调头区域的几何关系:

$$\begin{cases} R_1 = 2R_2 \\ 2R_1 \cos \theta_s + 2R_2 \cos \theta_s = 2R \end{cases} \quad (21)$$

然后我们联立 (20) 和 (21), 可以求出 $R_2 \approx 1.501655m$, 同时我们已知可以求出 S 点的坐标, 根据中心对称关系, 可以求出 F 点的坐标, 利用圆弧半径以及几何关系, 我们可以求出 $M(-0.761378, -1.307330)$, $N(1.736617, 2.449204)$, 大圆弧长 $9.143087m$, 小圆弧长 $4.571544m$, 此时的 S 形调头曲线长度为 $13.714630m$, 从而我们实现了对整个调头区域及调头路径的方程解析, 便于满足后续更具体分析的参数值需求。

5.4.2 调头有效区域不同时最短调头曲线的求解

在上一部分中我们给出了恰好在问题 3 所示直径为 $9m$ 的圆形调头空间的边界上开始调头时的相关参数求解, 总览问题 4 题目给出的各种约束条件之后, 我们或许会认为调头空间内 S 形调头曲线的位置及大小都是固定的, 其实不然, 我们不仅可以在调头空间的边界上立即开始调头, 我们还可以在沿等距螺线进一步向圆形调头空间内部延伸后, 在调头空间之内开始调头, 以螺线中心为原点, 调头开始点与原点之间的距离

为半径，画出的圆为实际有效的调头区域边界¹，在有效调头区域外部时，沿等距螺线进行行进，与问题 1 相似，在有效调头区域内部时的情况则相对复杂，具体可见下一部分中对路径段的划分，显然，随着有效调头区域逐渐缩小，内部的调头曲线长度也会减小，但不会无限减小，这是因为小到一定程度时会导致龙头路径卡死或者板凳相撞，也即，调整圆弧，仍保持各部分相切，我们可以使得调头曲线变短。尽管可以继续减小有效圆的半径，求出保证相切关系的理论上最小的有效圆半径，但在这种情况下板凳龙在调头路径上会卡死或相撞，这样就无从谈起后续要求解的-100s 到 100s 时间段内的各把手位置及速度了，所以实际可行的最短调头曲线需要保证既不会相撞也不会卡死。同上一部分对大圆弧与小圆弧长度的求解过程，我们可以改变调头区域有效圆半径，求出刚好不卡死下的调头区域有效圆半径为 $4.29m$ ，此时计算出的 S 形调头曲线的最短长度为 $13.059333m$ ，此部分求解的最短路径情况与 5.4.1 部分的可视化对比结果见图 12。

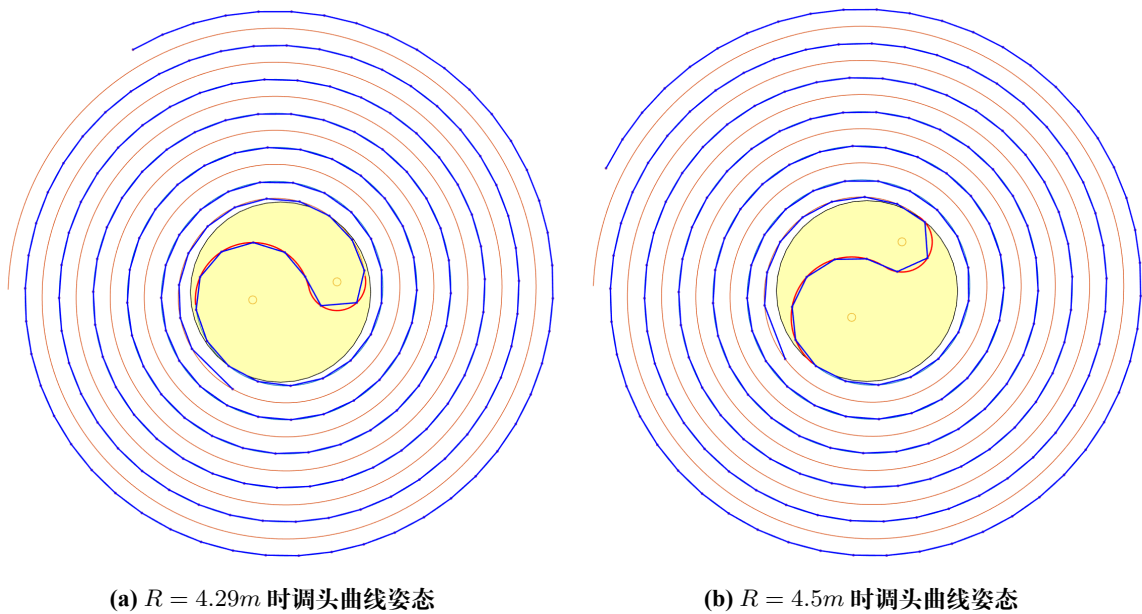


图 12 最短时与未调整之前的调头曲线姿态对比

5.4.3 调头过程不同情况分界点的求解

与盘入调头区域之前轨迹不同，调头过程的情况更为复杂，这是因为盘入调头区域之前，各个位置均为等距螺线，可以类似地进行迭代求解，调头过程的调头路径则涉及到三段曲线，即进入调头空间之前和从调头空间盘出的等距螺线，与盘入螺线相切的大圆弧，与盘出螺线相切的小圆弧，同时两段圆弧也是相切的，尽管三段曲线交接位置进行了平滑的过渡，但当两个把手点位于不同曲线上时，得出的迭代关系有很大的不同，从盘入调头到盘出过程中不同情况的临界点如图 13 所示。

¹注意，若无特别说明，从本页开始示意图中黄色圆形区域均代表实际有效圆调头区域

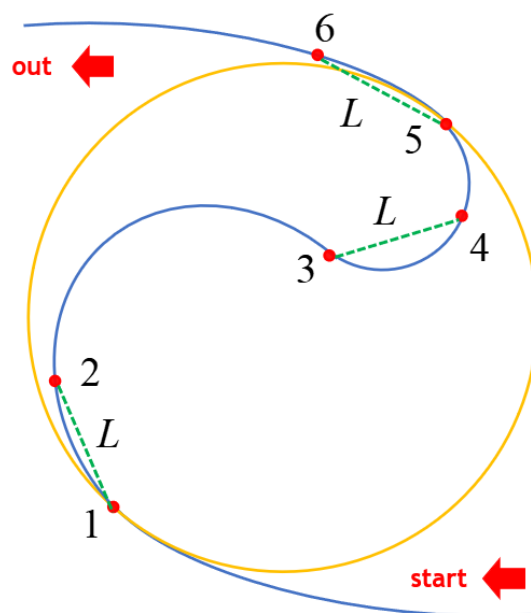


图 13 不同段情况临界点示意图

其中，龙头前把手中心点在到达点 1 之前，处于等距螺线上，与问题 1 类似，点 1 即为盘入时与调头空间的交点，经过点 1 后，板凳处于前把手在大圆弧上，后把手在等距螺线上的情况，直至到达点 2，点 2 可由以点 1 为圆心，以板凳前后把手之间的距离作为半径，画出的圆与大圆弧的交点来确定，经过点 2 后，板凳前后把手均在大圆弧上，直至点 3，点 3 即为两段圆弧之间相切的点，经过点 3 后，板凳处于前把手在小圆弧上，后把手在大圆弧上的情况，直至点 4，点 4 可由以点 3 为圆心，以板凳前后把手之间的距离作为半径，画出的圆与小圆弧的交点来确定，产生的与大圆弧相交的点要舍去，经过点 4 后，板凳前后把手均在小圆弧上，直至点 5，点 5 即为盘出时与调头空间的交点，直至点 6，点 6 可由以点 5 为圆心，以板凳前后把手之间的距离作为半径，画出的圆与盘出的等距螺线的交点来确定，产生的与小圆弧相交的点要舍去，依据这些划分出来的临界点，我们可以将调头过程分为 7 段情况。

5.4.4 调头过程各时刻各把手中心点位置与速度的求解

由上文分析可知，6 个临界点将调头路径按顺序分为 7 种不同的路径段，这些路径段各有各的不同，需要具体分析求解，各分段情况下的板凳姿态如图 14 所示，注意这里的外接圆表示的是有效圆而非完整的调头空间。

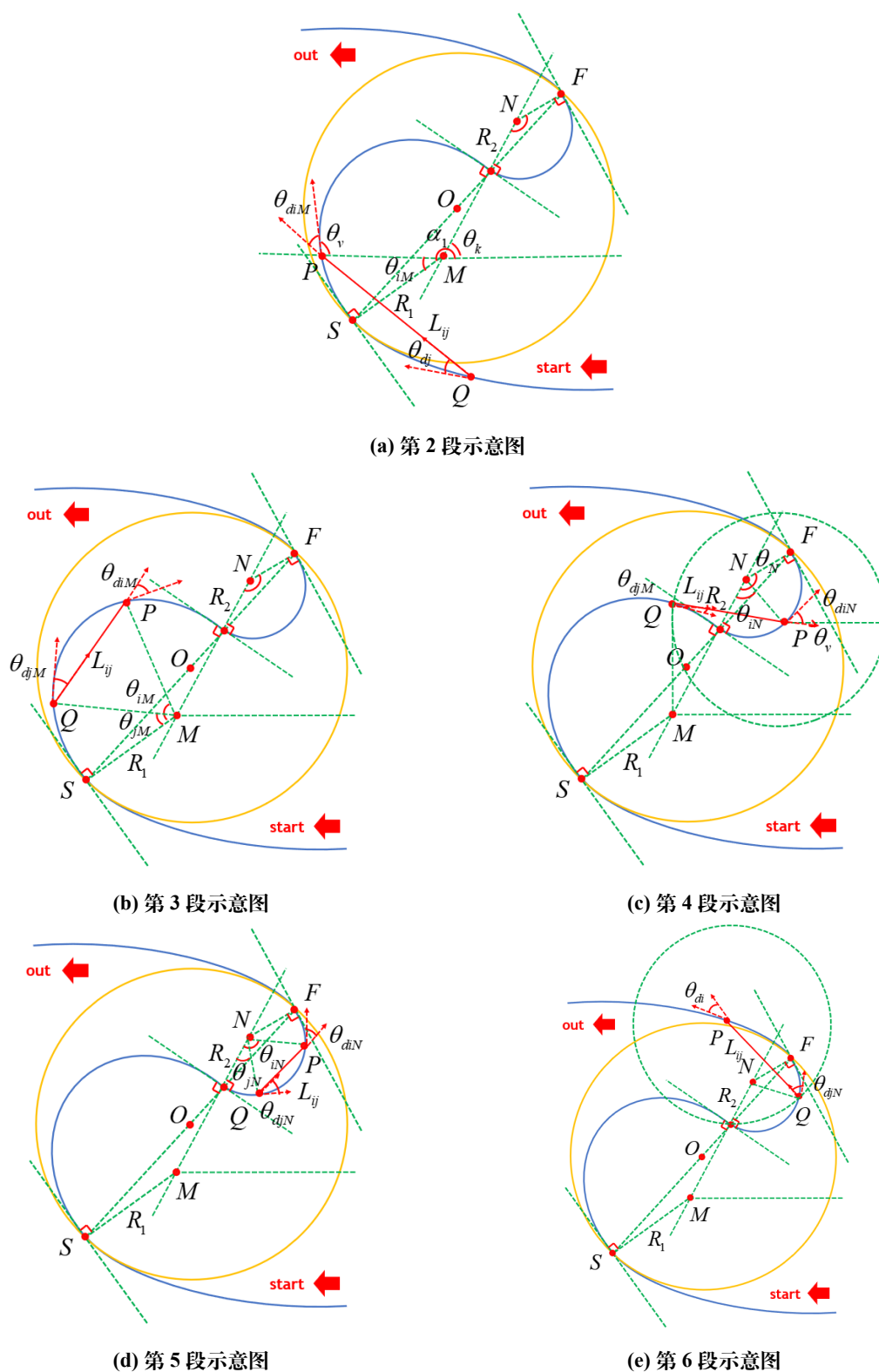


图 14 分路径 2~6 时板凳的姿态

由于 7 个分段情况中第 1 段和第 7 段时，板凳前后把手中心点均位于阿基米德等距螺线上，故这两种情况本质上类似于问题 1，我们使用类似的求解方法即可求解，除了这 2 种情况外，其余 5 种情况均需要具体分析求解，我们将在下面内容中逐步求解。

对于第 2 段情况，如图 14a 所示，板凳处于前把手在大圆弧上，后把手在等距螺线上的情况，由于我们在 5.4.1 中完成了对于本问题调头区域的基础参数求解，我们可以根据两个圆弧的圆心坐标结合平面几何关系来计算一些后续有用的角度值：

$$\begin{cases} \theta_k = \arctan \frac{y_N - y_M}{x_N - x_M} \\ \alpha_1 = \theta_k + \pi \end{cases} \quad (22)$$

其中， θ_k 表示直线 MN 与 x 轴正方向的夹角，计算 α_1 是为了后续根据圆弧圆心坐标计算把手中心点的坐标的方便，利用三角函数关系以及弧度的相关性质，我们可以得到：

$$\begin{cases} \theta_{iM} = \omega t = \frac{v_i}{R_1} t \\ (x_i, y_i) = (x_M + R_1 \cos(\alpha_1 - \theta_{iM}), y_M + R_1 \sin(\alpha_1 - \theta_{iM})) \end{cases} \quad (23)$$

其中 θ_{iM} 表示从进入调头区域开始到 t 时刻转过的角度，即图 14a 中 MS 与 MP 之间的夹角大小，我们将 α_1 与 θ_{iM} 相减得到了 MP 相对 x 轴正方向的角度，利用这个角度与大圆弧圆心 M 坐标表示出了点 P 的直角坐标，由于其它参数在给定要求下是固定的，故此坐标本质上可以看成关于时间 t 的函数，也可看做关于角度 θ_{iM} 的函数，我们从而获得了第 2 段下 P 点的位置坐标随时间的变化关系，于是我们就可迭代求解本段其他把手点的位置坐标，而板凳前后把手中心点间距是不可伸缩刚性的，表示为 L_{ij} ，所以：

$$(x_i - r_j \cos \theta_j)^2 + (y_i - r_j \sin \theta_j)^2 = L_{ij}^2 \Rightarrow Q : (r_j, \theta_j) \Rightarrow Q : (x_j, y_j) \quad (24)$$

也就是说，我们在第 2 段上可以实现递归求解各把手中心点的位置坐标，上式展示过程中首先计算出极坐标值，然后利用坐标变换关系得到了直角坐标值。

在计算出位置坐标递推关系后，我们可利用它来求解速度递推关系，如图 14a 所示，与问题 1 中一开始对各把手点的速度的求解相同，我们在这里同样使用速度分解的方法，使得前后两把手中心点速度沿它们之间的连线方向的速度分量相同，由于前后两个速度迭代求解时由一方确定另一方，所以问题转化为对速度方向与板凳前后把手连线方向角度的求解，利用已求出的前后把手坐标，我们可以间接计算出板凳方向与 x 轴正方向的相对角度，再利用圆弧上推导出的行进速度方向与 x 轴正方向的相对角度 θ_v ，相减即可得到大圆弧上速度方向与板凳方向的角度 θ_{diM} ，而等距螺线上该角度 θ_{dj} 推导过程与问题 1 相同，见 (15)，从而得到该段的速度递推关系：

$$\begin{cases} \theta_{diM} = \pi - \arctan \frac{y_i - y_j}{x_i - x_j} - \theta_v \\ \theta_v = \theta_k - \theta(t) + \frac{\pi}{2} \\ v_i \cos \theta_{diM} = v_j \cos \theta_{dj} \end{cases} \quad (25)$$

对于第 3 段情况，如图 14b 所示，板凳前后把手中心点均在大圆弧上，可以看到 P、Q 两点与大圆弧圆心的连线构成了一个三角形，该三角形三边大小已知，所以我们可以

使用余弦定理来求解出对应的圆心角，而在上一段中我们已经得知大圆弧上的点位置坐标可表示为关于 θ_{iM} 的函数，所以我们可以得出 Q 点坐标，从而能够迭代求解该段各把手中心点的位置坐标。

$$\begin{cases} \theta_{jM} = \theta_{iM} - \arccos \frac{2R_1^2 - L_{ij}^2}{2R_1^2} \\ (x_j, y_j) = (x_M + R_1 \cos(\alpha_1 - \theta_{jM}), y_M + R_1 \sin(\alpha_1 - \theta_{jM})) \end{cases} \quad (26)$$

第 3 段的速度递推关系的求解与上一分段情况类似，角度求解方式也与上一分段情况相同。

$$v_i \cos \theta_{diM} = v_j \cos \theta_{djM} \quad (27)$$

对于第 4 段情况，如图 14c 所示，板凳前把手点位于小圆弧上，后把手点位于大圆弧上，同理，我们可以得到：

$$\begin{cases} \theta_{iN} = \omega(t - \Delta t) = \frac{v_i}{R_2}(t - \Delta t) \\ (x_i, y_i) = (x_N + R_2 \cos(\alpha_1 + \theta_{iN}), y_N + R_2 \sin(\alpha_1 + \theta_{iN})) \end{cases} \quad (28)$$

要想根据 P 点的坐标求解 Q 点坐标，我们可以以 P 点为圆心， L_{ij} 为半径，画圆使圆与大圆弧相交，联立下式，从而得到 Q 点的位置坐标，注意这里我们可能会有两个交点，需要舍弃不合理的横坐标偏大的位置坐标解，从而本段上也可根据前方把手点位置坐标结合几何关系实现后方把手点位置坐标的计算，从而实现递归求解。

$$\begin{cases} (x - x_i)^2 + (y - y_i)^2 = L_{ij}^2 \\ (x - x_M)^2 + (y - y_M)^2 = R_1^2 \end{cases} \Rightarrow Q : (x_j, y_j), \text{omit bigger } x \quad (29)$$

第 4 段调头路径的速度求解也与前几个路径段类似，不同之处在于第 4 段时板凳前把手点已经处于小圆弧上，小圆弧上的 θ_v 推导结论与大圆弧上略有不同，其中 $\theta(N)$ 为 N 点处的圆心角，可以得出：

$$\begin{cases} \theta_{diN} = \theta_v + \arctan \frac{y_j - y_i}{x_j - x_i} \\ \theta_v = \theta_k + \theta(t) + \frac{\pi}{2} - \theta(N) \\ v_i \cos \theta_{diN} = v_j \cos \theta_{djM} \end{cases} \quad (30)$$

对于第 5 段情况，如图 14d 所示，板凳前后把手点均位于小圆弧上，位置坐标解法同第 3 段情况，可以实现迭代求解：

$$\begin{cases} \theta_{jN} = \theta_{iN} - \arccos \frac{2R_2^2 - L_{ij}^2}{2R_2^2} \\ (x_j, y_j) = (x_N + R_2 \cos(\alpha_1 - \theta_{jN}), y_N + R_2 \sin(\alpha_1 - \theta_{jN})) \end{cases} \quad (31)$$

第 5 段的速度递推关系求解也与第 3 段类似，不同之处在于从大圆弧变成了小圆弧，可以得出：

$$v_i \cos \theta_{diN} = v_j \cos \theta_{djN} \quad (32)$$

对于第 6 段情况，如图 14e 所示，板凳前把手点均位于向外盘出的等距螺线上，后把手点位于小圆弧上，由于盘入盘出点中心对称关系，该过程与盘入过程正好相反，将起初的盘入过程各把手点坐标中心对称转换即可得该段 P 点的坐标，为得到 Q 点的坐标，我们可以以 P 点为圆心， L_{ij} 为半径，画圆使圆与小圆弧相交，联立下式从而得解，注意这里圆可能会与小圆弧有两个交点，我们需要舍弃掉 x 值较小的解，从而在该段完成递归位置求解。

$$\begin{cases} (x - x_i)^2 + (y - y_i)^2 = L_{ij}^2 \\ (x - x_N)^2 + (y - y_N)^2 = R_2^2 \end{cases} \Rightarrow Q : (x_j, y_j), \text{omit smaller } x \quad (33)$$

本段的速度迭代关系与第 2 段类似，注意这里的 θ_{di} 与第 2 段中的 θ_{dj} 类似，与问题 1 中推导结论相同，根据几何关系不难得到：

$$v_i \cos \theta_{di} = v_j \cos \theta_{djN} \quad (34)$$

我们在本部分开始时已经提到第 1 段和第 7 段情况仍处于等距螺旋线上，本质上与问题 1 相同，至此，我们得到了调头空间内的所有分段的位置递推关系和速度递推关系，从而实现了各时刻各把手中心点的位置坐标和速度大小的计算。以调头开始时间为零时刻，从 -100s 开始到 100s 为止，调头区域有效圆半径等于 4.5m 时的每秒舞龙队的位置和速度完整结果见附件 **result4.xlsx**，这里只分有效圆半径等于 4.5m 和 4.29m 两种情况即初始情况和最短调头曲线情况展示部分结果，如表 4、表 5、表 6、表 7 所示。

表 4 盘入及调头过程部分时刻部分把手点的速度大小 ($R = 4.5m$)

	-100 s	-50 s	0 s	50 s	100 s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999904	0.999762	0.998687	1.000363	1.000124
第 51 节龙身 (m/s)	0.999346	0.998642	0.995134	0.722265	1.003966
第 101 节龙身 (m/s)	0.999091	0.998248	0.994448	0.721160	0.754402
第 151 节龙身 (m/s)	0.998944	0.998047	0.994156	0.720823	0.753744
第 201 节龙身 (m/s)	0.998849	0.997925	0.993994	0.720659	0.753487
龙尾 (后) (m/s)	0.998817	0.997885	0.993944	0.720612	0.753418

表 5 盘入及调头过程部分时刻部分把手点的位置坐标 ($R = 4.5m$)

	-100 s	-50 s	0 s	50 s	100 s
龙头 x (m)	7.776468	6.607374	-2.711856	1.433709	-3.059186
龙头 y (m)	3.720720	1.902637	-3.591078	6.148058	7.585045
第 1 节龙身 x (m)	6.206635	5.364540	-0.063534	3.940569	-0.242249
第 1 节龙身 y (m)	6.111373	4.478480	-4.670888	4.771377	8.079479
第 51 节龙身 x (m)	-10.609131	-3.626391	2.459962	-1.484350	2.184581
第 51 节龙身 y (m)	2.827766	-8.965355	-7.778145	-6.133709	3.978877
第 101 节龙身 x (m)	-11.921386	10.127833	3.008493	-7.704801	-7.358844
第 101 节龙身 y (m)	-4.806008	-5.968952	10.108539	5.016271	1.833912
第 151 节龙身 x (m)	-14.350573	12.975950	-7.002789	-4.427994	9.550023
第 151 节龙身 y (m)	-1.984847	-3.806659	10.337482	-10.468787	-3.316668
第 201 节龙身 x (m)	-11.955561	10.525334	-6.872842	0.532035	8.355881
第 201 节龙身 y (m)	10.564134	-10.804770	12.382609	-13.175182	8.778006
龙尾 (后) x (m)	-1.007190	0.185934	-1.933627	5.683203	-10.856541
龙尾 (后) y (m)	-16.527872	15.720701	-14.713128	12.697277	-6.975833

表 6 盘入及调头过程部分时刻部分把手点的速度大小 ($R = 4.29m, v_0 = 1m/s$)

	-100 s	-50 s	0 s	50 s	100 s
第 1 节龙身 (m/s)	0.999899	0.999742	0.998406	1.000394	1.000130
第 51 节龙身 (m/s)	0.999318	0.998554	0.994411	0.861062	1.004393
第 101 节龙身 (m/s)	0.999056	0.998147	0.993694	0.859657	0.871519
第 151 节龙身 (m/s)	0.998907	0.997941	0.993394	0.859240	0.870721
第 201 节龙身 (m/s)	0.998810	0.997817	0.993229	0.859041	0.870417
龙尾 (后) (m/s)	0.998778	0.997777	0.993177	0.858983	0.870335

表 7 盘入及调头过程部分时刻部分把手点的位置坐标 ($R = 4.29m$)

	-100 s	-50 s	0 s	50 s	100 s
龙头 x (m)	8.503244	6.575902	-4.243203	3.848418	-0.354628
龙头 y (m)	0.404571	-1.478886	-0.631924	4.852865	8.079089
第 1 节龙身 x (m)	7.979884	6.714871	-3.075946	5.514382	2.465027
第 1 节龙身 y (m)	3.216278	1.377736	-3.242884	2.528179	7.600398
第 51 节龙身 x (m)	-9.156992	-6.448366	-0.908127	-4.323471	3.941260
第 51 节龙身 y (m)	5.903550	-7.078200	-7.992968	-4.394446	1.889956
第 101 节龙身 x (m)	-12.694401	7.935286	6.014363	-5.327171	-5.708068
第 101 节龙身 y (m)	-1.490698	-8.566630	8.557055	7.370754	4.804176
第 151 节龙身 x (m)	-14.353125	11.552664	-3.935683	-7.245190	7.824191
第 151 节龙身 y (m)	1.421043	-6.896135	11.771810	-8.653917	-6.255615
第 201 节龙身 x (m)	-9.400999	7.798198	-3.711331	-2.802049	10.410307
第 201 节龙身 y (m)	12.818375	-12.840123	13.599769	-12.813966	6.053722
龙尾 (后) x (m)	-4.359217	3.557124	-5.204960	8.522716	-12.239682
龙尾 (后) y (m)	-15.916514	15.253749	-13.830627	10.911633	-3.855506

5.5 问题五求解

本问题中舞龙队沿问题 4 设定的路径行进，调整不同的龙头行进速度，使得在行进过程中保持龙头前把手速度不变，其他各把手点的速度的最大值不超过 $2m/s$ ，我们需要求出满足这个条件的最大龙头前把手行进速度，而我们在问题 4 中已经求出了盘入、调头、盘出全过程各种情况的速度递推关系，该递推关系限制了特定时刻从龙头前把手到龙尾后把手所有把手点的速度大小，而随着时刻改变，除了龙头前把手之外，其他各把手点的速度均会随时间而改变，我们模拟了从起始点 $0s$ 开始，时间一直延续的情况下所有把手点的速度变化趋势，我们可以得出，随着时间延续，各时刻所有把手点速度中最大值随时间变化的趋势 $v_{max}(t)$ 类似于开口向下的二次函数，也就是说，在固定龙头前把手点速度的情况下，存在某个时刻 t_m ，使得达到所有把手点速度能达到的最大

值的顶峰 $v_{max}(t_m)$ ，该值需要不超过 $2m/s$ ，从而求得最大龙头前把手行进速度，整个流程可看作如下的优化模型：

$$\max v_0 \Rightarrow v_{0m} \quad \text{s.t.} \quad \begin{cases} \exists t_m, \forall t, v_{max}(t) \leq v_{max}(t_m), \\ \forall v_0 \leq v_{0m}, \forall v_{max}(t_m) \leq 2m/s, \\ v_j = f(v_i), f : v_j \rightarrow v_i. \end{cases} \quad (35)$$

在具体迭代求解过程中，与问题 3 类似，我们只需要使用基础的固定步长逐渐增加龙头前把手行进速度，据此来观察 $v_{max}(t_m)$ 的变化，使得 v_0 在满足约束下达到最大，即为所求，具体细节见附件 *matlab* 代码文件。最终我们求解出，当 $R = 4.29m$ 时，龙头最大行进速度为 $0.6313m/s$ ，所有把手点所有时刻的最大速度为 $1.9908m/s$ ，当龙头速度为 $0.6313m/s$ 时，所有把手的最大速度与时间的关系趋势见图 15， $R = 4.5m$ 时，龙头最大行进速度为 $1.232m/s$ ，所有把手点所有时刻的最大速度为 $1.9998m/s$ ，当龙头速度为 $1.232m/s$ 时，所有把手的最大速度与时间的关系趋势见图 16。

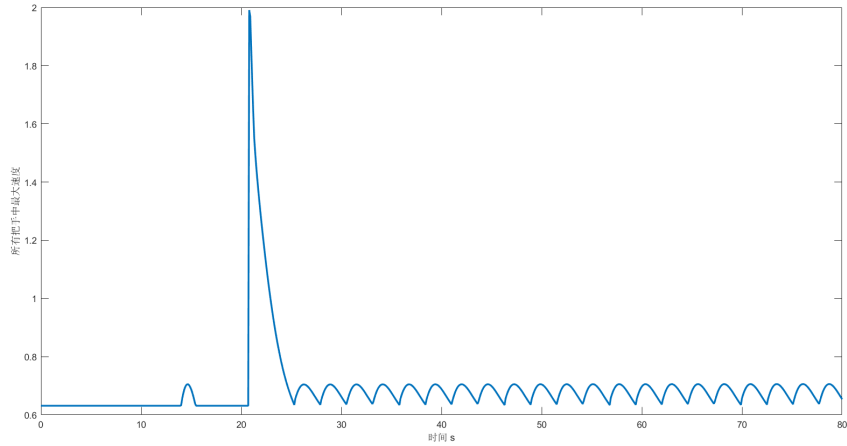


图 15 龙头速度为 $0.6313m/s$ 时，所有把手的最大速度与时间的关系图

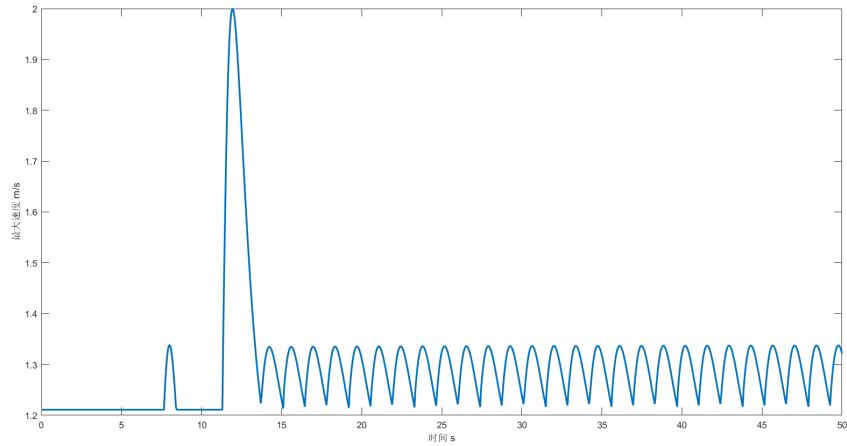


图 16 龙头速度为 $1.232m/s$ 时，所有把手的最大速度与时间的关系图

六、模型结果检验与分析

6.1 问题 1 推导出的 $\theta(t)$ 的数值解的误差检验

由于龙头前把手点的行进速度 $v_0 = 1\text{m/s}$ ，所以 1s 后行进的弧长为 1m，我们将 1s 代入 $\theta(t)$ 的数值解求解过程可以得出 θ ，然后再代入阿基米德螺旋的弧长公式 [4] 得到此段时间行进弧长大小：

$$s(\theta) = \frac{1}{2}a[\theta\sqrt{1+\theta^2} + \ln(\theta + \sqrt{1+\theta^2})] \quad (36)$$

其中，根据 (6) 的第一个式子，我们可以知道 $a = \frac{d}{2\pi}$ ，利用 (36) 我们计算出 1s 内行进弧长与 1m 只差 $1e^{-6}$ 数量级的误差，从而证明了对 $\theta(t)$ 的推导过程与数值解求解结果是正确的，只有极小的误差且是可以接受的。

6.2 问题 5 速度求解逻辑的可视化检验

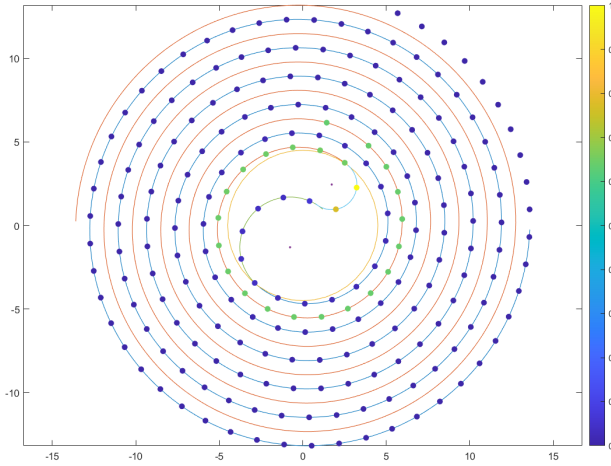


图 17 速度大小与位置关系图

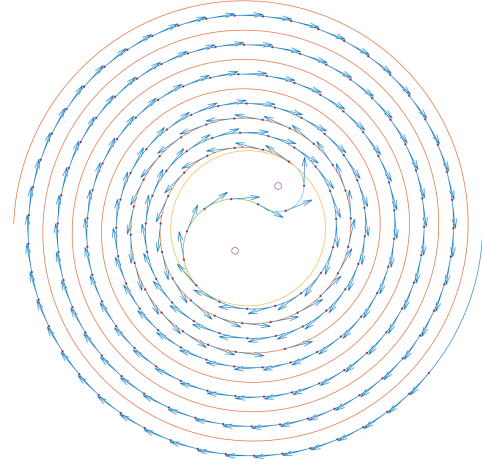


图 18 速度方向与位置关系图

图 17 是我们得出的用颜色深浅表示的速度大小与位置关系图，根据图像可知，小圆是速度发生剧变的区域，因此当龙头进入小圆时，后端所有点的速度将发生非常剧烈的变化，这与问题 5 的最大速度与时间的关系图出现尖峰的情况符合得很好，间接验证了其正确性。图 18 是我们得出的用有向箭头表示的速度方向与位置关系图，我们也可以从此图窥得各把手速度牵制关系与递推逻辑的合理性，与直觉认知相符合，实际求解结果也合乎预判。

七、模型优缺点分析

7.1 模型优点分析

- ①**模型结果精度较高**。对于各个把手的求解均根据关于圆心角的微分方程求解，而非采用对螺线轨迹长度的近似估计算法，不会产生较大累计误差。
- ②**求解过程严谨**。模型中的数学表达式都有明确的几何数学以及物理定理作为依据。
- ③**算法有合理的优化，计算复杂度低**。例如在计算问题 2、3 的碰撞位置及时间的过程中，对于可能发生碰撞的板凳进行筛选，大大减少了计算量。
- ④**模型结果易于理解，可解释性高**。模型给出的结果能很好用数学公式来解释，且能够直观体现盘龙的形态。

7.2 模型缺点分析

- ①**在少许算法存在过度拟合**。例如第四问的方法只适用于调头区域为两段圆弧的情况，假如为其他轨迹需要重新推导数学表达式。
- ②**计算过程仍有优化空间**。计算中存在多次极坐标与平面直角坐标的转化，可能导致计算时间增加，误差积累。

八、模型改进与推广

8.1 模型的进一步改进

本赛题的模型在求解问题 3 变螺距优化问题和问题 5 变行进速度优化问题时，只使用了最为基础的固定步长逐步变化参数值的优化方法来求解，因为对于本题的计算复杂度和本质上单参数的调节来说简单优化已经够用了，且能保证精度满足题目要求，但在面对同类型的现实问题时，涉及到的参数往往更多，计算复杂度往往更高，这就需要模型使用更为智能的优化算法，比如模拟退火算法、随机梯度下降法、遗传算法等启发式智能优化算法，本模型可以在替换优化算法层面进行改进。

8.2 模型的推广

在应用层面，本模型涉及的思想不仅适用于本赛题的求解优化，还适用于更广泛领域的多种同类型问题，比如机械工程中的凸轮结构问题、现代物流运输系统的链式运输问题、智能医疗领域的手术机器人的关节连杆调动问题等，只需要进行参变量的修改即可。在用户交互层面，本模型依据题目本身出发，主要功能以 *matlab* 代码的形式呈现，但本模型还可用作图像化交互界面的后端，以提供用户友好的交互。

九、参考文献

- [1] 白小虎. 文化内生制度与经济文化解释——鸡毛换糖、义乌兵与板凳龙 [J]. 浙江社会科学, 2006(2):116-122
- [2] <https://www.cnblogs.com/Tom-Ren/p/10234973.html>
- [3] 林文惠. 刚体运动学的几何讨论 [J]. 力学与实践, 2005, 27(5). DOI:10.6052/1000-0992-2004-405
- [4] <https://mathworld.wolfram.com/ArchimedesSpiral.html>

附录 A 支撑材料列表

文件类型	文件名称 L	文件名称 R
<i>Excel</i> 文件	<i>result1.xlsx</i>	
	<i>result2.xlsx</i>	
	<i>result4.xlsx</i>	
<i>Q1 Matlab</i> 文件	<i>getBenchDirection.m</i>	<i>getThetaByT.m</i>
	<i>getBodyLocation.m</i>	<i>getXYByTheta.m</i>
	<i>getDirectionByTheta.m</i>	<i>problem1.m</i>
<i>Q2 Matlab</i> 文件	<i>blank4Angles.m</i>	<i>getThetaByT.m</i>
	<i>getBenchDirection.m</i>	<i>getXYByTheta.m</i>
	<i>getBodyLocation.m</i>	<i>pointDetermine.m</i>
	<i>getDirectionByTheta.m</i>	<i>problem2.m</i>
<i>Q3 Matlab</i> 文件	<i>blank4Angles.m</i>	<i>getThetaByT.m</i>
	<i>getBenchDirection.m</i>	<i>getXYByTheta.m</i>
	<i>getBodyLocation.m</i>	<i>pointDetermine.m</i>
	<i>getDirectionByTheta.m</i>	<i>problem3.m</i>
<i>Q4 Matlab</i> 文件	<i>blank4Angles.m</i>	<i>getThetaByT.m</i>
	<i>getBenchDirection.m</i>	<i>getXY.m</i>
	<i>getBodyLocation.m</i>	<i>iterHandles4.m</i>
	<i>getDirectionByTheta.m</i>	<i>problem4.m</i>
	<i>getSolution.m</i>	<i>problem4Plus.m</i>
<i>Q5 Matlab</i> 文件	<i>problem5.m</i>	

附录 B 问题 1 Matlab 代码

getBenchDirection.m:

% 输入两个点的极坐标，输出两个点向量与x轴正方向的夹角，即板凳方向

```
function theta = getBenchDirection(r0, theta0, r1, theta1)
```

```
    dx = r0*cos(theta0) - r1*cos(theta1);
```

```
    dy = r0*sin(theta0) - r1*sin(theta1);
```

```
    theta = atan2(dy, dx);
```

```
    if theta < 0
```

```
        theta = theta + 2*pi;
```

```
    end
```

```
end
```

getBodyLocation.m:

% 输入龙头相角，输出整条龙包括龙头、龙身、龙尾的位置（极坐标）

```
function [R, THETA] = getBodyLocation(head_theta)
```

```
    dr0 = 1;
```

```
    sum = 223;
```

```
    k = 0.55/(2*pi);
```

```
    r0 = head_theta * k;
```

```
    % 板凳孔径距离
```

```
    head_inner_length = 2.86;
```

```
    body_inner_length = 1.65;
```

```
    R = zeros(1, sum+1);
```

```
    THETA = zeros(1, sum+1);
```

```
    f = @(x) (x^2+r0^2-head_inner_length^2)-2*x*r0*cos((x-r0)/k);
```

```
    % 龙头
```

```
    R(1) = r0;
```

```
    THETA(1) = R(1)/k;
```

```
    % 第一个龙身前把手
```

```
    R(2) = fzero(f, [r0, r0+dr0]);
```

```
    THETA(2) = R(2)/k;
```

```
    % 其余把手
```

```
    for i = 3:sum+1
```

```
        r0 = R(i-1);
```

```
        f = @(x) (x^2+r0^2-body_inner_length^2)-2*x*r0*cos((x-r0)/k);
```

```
        R(i) = fzero(f, [R(i-1), R(i-1)+dr0]);
```

```
        THETA(i) = R(i)/k;
```

```
    end
```

```
end
```

getDirectionByTheta.m:

```
% 通过theta角计算当前速度方向与x轴正方向的夹角
% 参数:
% theta: 从0开始, 由内到外的角, 大于0
% inOrOut: 盘内还是盘外 0盘内, 1盘外
function resAngle = getDirectionByTheta(theta,inOrOut)
    if inOrOut
        resAngle = atan(theta) + theta;
    else
        resAngle = atan(theta) + theta + pi;
    end
end
```

getThetaByT.m:

```
function result = getThetaByT(tfinal)
    b = 32 * pi;
    start_t = (b*sqrt(b^2+1)+log(b+sqrt(b^2+1)))*0.55/(4*pi);
    % 从进入螺线开始计时
    % 提高解的精度
    options = odeset('RelTol',1e-10,'AbsTol',1e-10,'InitialStep', 1e-5, 'MaxStep', 1);
    % 求偏微分方程的离散解
    [t, Xt] = ode45(@SunFun, [0 start_t-tfinal], 0, options);
    result = Xt(end);

    % 微分方程函数, 状态导数
    function xdot = SunFun(t, x)
        xdot = 2*pi/0.55*1/sqrt(x^2+1);
    end
end
```

getXYByTheta.m:

```
function [x, y, r] = getXYByTheta(theta)

    r = 0.55/(2*pi)*theta;
    x = r*cos(theta);
    y = r*sin(theta);

end
```

problem1.m:

```
pitch = 0.55/(2*pi);
theta0 = 0:0.01*pi:32*pi;
r1 = pitch * theta0;
% 板凳数据
% hole_distance = 27.5;
```



```

% head_length = 341;
% head_width = 30;
% head_inner_length = head_length - 2*hole_distance; % 孔径距离
% body_length = 220;
% body_width = 30;
% body_inner_length = body_length - 2*hole_distance;

XY = zeros(448, 301);
VV = zeros(224, 301);

for t = 0:300
    fprintf('%d',t);
    theta = getThetaByT(t);
    [x,y,r] = getXYByTheta(theta);
    [R,THETA] = getBodyLocation(theta);

    % 获取速度
    lastR = R(1);
    lastTheta = THETA(1);
    lastV = 1;
    V = zeros(1,224);
    V(1) = lastV;
    XY_row = zeros(448,1);
    XY_row(1) = x;
    XY_row(2) = y;
    for i = 2:length(R)
        r = R(i);
        theta = THETA(i);
        alpha = getDirectionByTheta(lastTheta,0);
        beta = getDirectionByTheta(theta,0);
        omega = getBenchDirection(lastR, lastTheta, r, theta);
        v = lastV*cos(omega-alpha)/cos(beta-omega);
        V(i) = v;
        lastV = v;
        lastR = R(i);
        lastTheta = THETA(i);

        % 获取x和y
        [x,y,r] = getXYByTheta(theta);
        XY_row(i*2-1) = x;
        XY_row(i*2) = y;
    end
    % 表格数据
    XY(:,t+1) = XY_row;
    VV(:,t+1) = V';
end

```

```
% 画图
polaraxes;
polarscatter(THETA, R, 20, 'filled');hold on;
polarscatter(THETA(1), R(1), 20, 'filled', 'r');hold on;
polarplot(theta0,r1,'b');
```

附录 C 问题 2 Matlab 代码

blank4Angles.m:

```
% 根据两个把手的坐标 (xy)，获得四个角上点的坐标 (xy)
% 输入：两个把手的平面直角坐标,椅子总长度
% 输出：四个点，resp1 resp2为短边两点，resp2,resp3为长边两点
function [resp1,resp2,resp3,resp4] = blank4Angles(p0,p1,length)
    width = 0.3; % 板凳宽度
    k = (p0(2)-p1(2))/(p0(1)-p1(1));
    centerX = (p0(1)+p1(1))/2;
    centerY = (p0(2)+p1(2))/2;
    % 接下来求长边的中点坐标
    deltaX = length/2*(1/sqrt(1+k^2));
    deltaY = length/2*(k/sqrt(1+k^2));
    pm1 = [centerX+deltaX centerY+deltaY];
    pm2 = [centerX-deltaX centerY-deltaY];
    k2 = -1/k; % 中垂线
    deltaX = width/2*(1/sqrt(1+k2^2));
    deltaY = width/2*(k2/sqrt(1+k2^2));
    resp1 = [pm1(1)-deltaX pm1(2)-deltaY];
    resp2 = [pm1(1)+deltaX pm1(2)+deltaY];
    resp3 = [pm2(1)+deltaX pm2(2)+deltaY];
    resp4 = [pm2(1)-deltaX pm2(2)-deltaY];
end
```

getBenchDirection.m:

```
% 输入两个点的极坐标，输出两个点向量与x轴正方向的夹角，即板凳方向
function theta = getBenchDirection(r0, theta0, r1, theta1)

    dx = r0*cos(theta0) - r1*cos(theta1);
    dy = r0*sin(theta0) - r1*sin(theta1);
    theta = atan2(dy, dx);
    if theta < 0
        theta = theta + 2*pi;
    end
end
```

getBodyLocation.m:

```
% 输入龙头相角，输出整条龙包括龙头、龙身、龙尾的位置（极坐标）
function [R, THETA] = getBodyLocation(head_theta)

    dr0 = 1;
    sum = 223;
    k = 0.55/(2*pi);
    r0 = head_theta * k;
    % 板凳孔径距离
    head_inner_length = 2.86;
    body_inner_length = 1.65;
    R = zeros(1,sum+1);
    THETA = zeros(1,sum+1);

    f = @(x) (x^2+r0^2-head_inner_length^2)-2*x*r0*cos((x-r0)/k);

    % 龙头
    R(1) = r0;
    THETA(1) = R(1)/k;
    % 第一个龙身前把手
    R(2) = fzero(f,[r0,r0+dr0]);
    THETA(2) = R(2)/k;

    % 其余把手
    for i = 3:sum+1
        r0 = R(i-1);
        f = @(x) (x^2+r0^2-body_inner_length^2)-2*x*r0*cos((x-r0)/k);
        R(i) = fzero(f,[R(i-1),R(i-1)+3]);
        THETA(i) = R(i)/k;
    end

end
```

getDirectionByTheta.m:

```
% 通过theta角计算当前速度方向与x轴正方向的夹角
% 参数：
% theta: 从0开始，由内到外的角，大于0
% inOrOut: 盘内还是盘外 0盘内，1盘外
function resAngle = getDirectionByTheta(theta,inOrOut)

    if inOrOut
        resAngle = atan(theta) + theta;
    else
        resAngle = atan(theta) + theta + pi;
    end

end
```

getThetaByT.m:

```
function result = SunFun(tfinal)

    b = 32 * pi;
    start_t = (b*sqrt(b^2+1)+log(b+sqrt(b^2+1)))*0.55/(4*pi);
    % 从进入螺线开始计时
    % 提高解的精度
    options = odeset('RelTol',1e-10,'AbsTol',1e-10);
    % 求偏微分方程的离散解
    [t, Xt] = ode45(@SunFun, [0 start_t-tfinal], 0, options);

    result = Xt(end);

    % 微分方程函数，状态导数
    function xdot = SunFun(t, x)
        xdot = 2*pi/0.55*1/sqrt(x^2+1);
    end
end
```

getXYByTheta.m:

```
function [x, y, r] = getXYByTheta(theta)

    r = 0.55/(2*pi)*theta;
    x = r*cos(theta);
    y = r*sin(theta);
end
```

pointDetermine.m:

```
% 判断点是否在矩形内
% 1为是，0为否
function result = pointDetermine(points, p)

    result = 1;
    % 宽判断
    p1 = points(1,:);
    p2 = points(2,:);
    if dot(p1-p,p1-p2)<0 || dot(p2-p,p2-p1)<0
        result = 0;
    end
    % 长判断
    p1 = points(2,:);
    p2 = points(3,:);
    if dot(p1-p,p1-p2)<0 || dot(p2-p,p2-p1)<0
        result = 0;
    end
end
```

problem2.m:

```
% 曼哈顿距离阈值
maxDistance = 5;
maxRadiusDifference = 1;
body_length = 2.2;
head_length = 3.41;
bars = 30;

% 判断是否达成碰撞条件的标志位, 碰撞时间点约为412.474054
flag = 0;
for t = 412.474:0.000001:412.4742
% for t = 413.5:0.01:413.6
    theta = getThetaByT(t);
    [x0,y0,~] = getXYByTheta(theta);
    [R,THETA] = getBodyLocation(theta);

    for bar = 1:bars
        if bar == 1
            l = head_length;
        else
            l = body_length;
        end
        [x0,y0,~] = getXYByTheta(THETA(bar));
        % 获得目标板凳的四点
        [x1,y1,~] = getXYByTheta(THETA(bar+1));
        [a0,b0,c0,d0] = blank4Angles([x0,y0],[x1,y1],l);
        P = [a0;b0;c0;d0];
        % 取出目标板凳最有可能碰撞的两个点 (即靠外的点)
        [~,indexs] = sort([norm(a0),norm(b0),norm(c0),norm(d0)]);
        p1 = P(indexs(end),:);
        p2 = P(indexs(end-1),:);

        p = [p1;p2];
        % 将两点分别进行碰撞检测
        for j = 1:2
            for i = bar+2:length(R)-1
                % 板凳的两个孔径坐标
                [x1,y1,~] = getXYByTheta(THETA(i));
                [x2,y2,~] = getXYByTheta(THETA(i+1));
                % 距离过远的板凳忽略
                if abs(p(j,1)-x1)+abs(p(j,2)-y1) > maxDistance ||
                    abs(norm([x0,y0])-norm([x1,y1])) > maxRadiusDifference
                    continue
                end

                % 判断是否碰撞
```

```

        [a,b,c,d] = blank4Angles([x1,y1],[x2,y2],body_length); % 板凳四点
        if pointDetermine([a;b;c;d],p(j,:))
%           figure
%           plot([a(1) b(1) c(1) d(1) a(1)],[a(2) b(2) c(2) d(2) a(2)]);hold on;
%           plot([p(j,1)],[p(j,2)],'.')
%           xlim([-10,10])
%           ylim([-10,10])
            flag=1;
            fprintf('发生碰撞');
            break
        end
    end
    if flag
        break
    end
end

    if flag
        break
    end
end
fprintf('%d\r',t);
if flag
    break
end
end
end

% 画出所有板凳
% 碰撞时刻
t_collide = t;
figure
xlim([-10,10])
ylim([-10,10])
for i = 1:223
    if i == 1
        l = head_length;
    else
        l = body_length;
    end
    [x1,y1,~] = getXYByTheta(THETA(i));
    [x2,y2,~] = getXYByTheta(THETA(i+1));
    [a,b,c,d] = blank4Angles([x1,y1],[x2,y2],l);
    plot([a(1) b(1) c(1) d(1) a(1)],[a(2) b(2) c(2) d(2) a(2)]);hold on;
    plot([p(j,1)],[p(j,2)],'.','Color','r','MarkerSize',10);hold on;
end

% 表格数据

```

```

X = zeros(224,1);
Y = zeros(224,1);
V = zeros(224,1);
theta = getThetaByT(t_collide);
[x0,y0,~] = getXYByTheta(theta);
[R,THETA] = getBodyLocation(theta);
lastR = R(1);
lastTheta = THETA(1);
lastV = 1;
% 初始值
X(1) = x0;
Y(1) = y0;
V(1) = 1;
for i = 2:224
    [x,y,~] = getXYByTheta(THETA(i));
    X(i) = x;
    Y(i) = y;
    % 获取速度
    r = R(i);
    theta = THETA(i);
    alpha = getDirectionByTheta(lastTheta,0);
    beta = getDirectionByTheta(theta,0);
    omega = getBenchDirection(lastR, lastTheta, r, theta);
    v = lastV*cos(omega-alpha)/cos(beta-omega);

    V(i) = v;
    lastV = v;
    lastR = R(i);
    lastTheta = THETA(i);
end

pitch = 0.55/(2*pi);
theta0 = 0:0.01*pi:32*pi;
r1 = pitch * theta0;
result = [X,Y,V];
% polarplot(theta0,r1,'b');
[x,y] = pol2cart(theta0,r1);
plot(x,y);
axis equal

```


附录 D 问题 3 Matlab 代码

blank4Angles.m:

```
% 根据两个把手的坐标 (xy)，获得四个角上点的坐标 (xy)
% 输入：两个把手的平面直角坐标,椅子总长度
% 输出：四个点，resp1 resp2为短边两点，resp2,resp3为长边两点
function [resp1,resp2,resp3,resp4] = blank4Angles(p0,p1,length)

    width = 0.3; % 板凳宽度
    k = (p0(2)-p1(2))/(p0(1)-p1(1));
    centerX = (p0(1)+p1(1))/2;
    centerY = (p0(2)+p1(2))/2;
    % 接下来求长边的中点坐标
    deltaX = length/2*(1/sqrt(1+k^2));
    deltaY = length/2*(k/sqrt(1+k^2));
    pm1 = [centerX+deltaX centerY+deltaY];
    pm2 = [centerX-deltaX centerY-deltaY];
    k2 = -1/k; % 中垂线
    deltaX = width/2*(1/sqrt(1+k2^2));
    deltaY = width/2*(k2/sqrt(1+k2^2));
    resp1 = [pm1(1)-deltaX, pm1(2)-deltaY];
    resp2 = [pm1(1)+deltaX, pm1(2)+deltaY];
    resp3 = [pm2(1)+deltaX, pm2(2)+deltaY];
    resp4 = [pm2(1)-deltaX, pm2(2)-deltaY];

end
```

getBenchDirection.m:

```
% 输入两个点的极坐标，输出两个点向量与x轴正方向的夹角，即板凳方向
function theta = getBenchDirection(r0, theta0, r1, theta1)

    dx = r0*cos(theta0) - r1*cos(theta1);
    dy = r0*sin(theta0) - r1*sin(theta1);
    theta = atan2(dy, dx);
    if theta < 0
        theta = theta + 2*pi;
    end

end
```

getBodyLocation.m:

```
% 输入龙头相角，输出整条龙包括龙头、龙身、龙尾的位置（极坐标）
function [R, THETA] = getBodyLocation(head_theta, screwPitch)

    dr0 = 3;
    sum = 223;
    k = screwPitch/(2*pi);
    r0 = head_theta * k;
    % 板凳孔径距离
    head_inner_length = 2.86;
    body_inner_length = 1.65;
    R = zeros(1,sum+1);
    THETA = zeros(1,sum+1);
    f = @(x) (x^2+r0^2-head_inner_length^2)-2*x*r0*cos((x-r0)/k);
    % 龙头
    R(1) = r0;
    THETA(1) = R(1)/k;
    % 第一个龙身前把手
    try
        R(2) = fzero(f, [r0, r0+dr0]);
    catch
        1
    end
    THETA(2) = R(2)/k;

    % 其余把手
    for i = 3:sum+1
        r0 = R(i-1);
        f = @(x) (x^2+r0^2-body_inner_length^2)-2*x*r0*cos((x-r0)/k);
        R(i) = fzero(f, [R(i-1), R(i-1)+dr0]);
        THETA(i) = R(i)/k;
    end
end
```

getDirectionByTheta.m:

```
% 通过theta角计算当前速度方向与x轴正方向的夹角
% 参数：
% theta: 从0开始，由内到外的角，大于0
% inOrOut: 盘内还是盘外 0盘内，1盘外
function resAngle = getDirectionByTheta(theta, inOrOut)

    if inOrOut
        resAngle = atan(theta) + theta;
    else
        resAngle = atan(theta) + theta + pi;
    end
end
```

getThetaByT.m:

```
function result = getThetaByT(tfinal, screwPitch)

    b = 32 * pi;
    start_t = (b*sqrt(b^2+1)+log(b+sqrt(b^2+1)))*screwPitch/(4*pi);
    % 从进入螺线开始计时
    % 提高解的精度
    options = odeset('RelTol',1e-10,'AbsTol',1e-10);
    % 求偏微分方程的离散解
    [t, Xt] = ode45(@SunFun, [0 start_t-tfinal], 0, options);

    result = Xt(end);

    % 微分方程函数，状态导数
    function xdot = SunFun(t, x)
        xdot = 2*pi/screwPitch*1/sqrt(x^2+1);
    end
end
```

getXYByTheta.m:

```
function [x, y, r] = getXYByTheta(theta, screwPitch)

    r = screwPitch/(2*pi)*theta;
    x = r*cos(theta);
    y = r*sin(theta);
end
```

pointDetermine.m:

```
% 判断点是否在矩形内
% 1为是，0为否
function result = pointDetermine(points, p)

    result = 1;
    % 宽判断
    p1 = points(1,:);
    p2 = points(2,:);
    if dot(p1-p,p1-p2)<0 || dot(p2-p,p2-p1)<0
        result = 0;
    end
    % 长判断
    p1 = points(2,:);
    p2 = points(3,:);
    if dot(p1-p,p1-p2)<0 || dot(p2-p,p2-p1)<0
        result = 0;
    end
end
```

problem3.m:

```
% 曼哈顿距离阈值
maxDistance = 5;
maxRadiusDifference = 1;
body_length = 2.2;
head_length = 3.41;
bars = 30;
space_r = 4.5;
minScrewPitch = 0.55;

for screwPitch = 0.45035:-0.00001:0.4503
    % 单次定螺距循环，判断是否达成碰撞条件的标志位
    flag = 0;
    for t = 215:0.1:1000
        theta = getThetaByT(t, screwPitch);
        [x0,y0,r0] = getXYByTheta(theta, screwPitch);
        [R,THETA] = getBodyLocation(theta, screwPitch);

        for bar = 1:bars
            if bar == 1
                l = head_length;
            else
                l = body_length;
            end
            [x0,y0,~] = getXYByTheta(THETA(bar), screwPitch);
            % 获得目标板凳的四点
            [x1,y1,~] = getXYByTheta(THETA(bar+1), screwPitch);
            [a0,b0,c0,d0] = blank4Angles([x0,y0],[x1,y1],l);
            P = [a0;b0;c0;d0];
            % 取出目标板凳最有可能碰撞的两个点（即靠外的点）
            [~,indexs] = sort([norm(a0),norm(b0),norm(c0),norm(d0)]);
            p1 = P(indexs(end),:);
            p2 = P(indexs(end-1),:);

            p = [p1;p2];
            % 将两点分别进行碰撞检测
            for j = 1:2
                for i = bar+2:length(R)-1
                    % 板凳的两个孔径坐标
                    [x1,y1,~] = getXYByTheta(THETA(i), screwPitch);
                    [x2,y2,~] = getXYByTheta(THETA(i+1), screwPitch);
                    % 距离过远的板凳忽略
                    if abs(p(j,1)-x1)+abs(p(j,2)-y1) > maxDistance ||
                        abs(norm([x0,y0])-norm([x1,y1])) > maxRadiusDifference
                        continue
                    end
                end
            end
        end
    end
end
```

```

        % 判断是否碰撞
        [a,b,c,d] = blank4Angles([x1,y1],[x2,y2],body_length); % 板凳四点
        if pointDetermine([a;b;c;d],p(j,:))
            flag=1;
            fprintf('\n螺距: %f发生碰撞。t=%f', screwPitch,t);
            break
        end
    end
    if flag
        break
    end
end

if flag
    break
end

end
fprintf('-');
if flag
    break
end

% 判断是否到达调头空间
if r0 <= space_r
    minScrewPitch = screwPitch;
    fprintf('\n螺距: %f可以到达调头空间。t=%f\n',screwPitch,t);
    break
end

end
if flag
    % 发生碰撞, 且没有到达调头空间, 说明该螺距及之后更小的螺距不满足情况, 跳出循环

    break
end
end
screwPitch = 0.44;
% 画出调头空间
theta = 0:0.01:2*pi;
x=4.5*cos(theta);
y=4.5*sin(theta);
fill(x,y,'y');hold on;
% 画出所有板凳
% 碰撞时刻
t_collide = t;
xlim([-10,10])
ylim([-10,10])

```

```

for i = 1:223
    if i == 1
        l = head_length;
    else
        l = body_length;
    end
    [x1,y1,~] = getXYByTheta(THETA(i), screwPitch);
    [x2,y2,~] = getXYByTheta(THETA(i+1), screwPitch);
    [a,b,c,d] = blank4Angles([x1,y1],[x2,y2],l);
    plot([a(1) b(1) c(1) d(1) a(1)],[a(2) b(2) c(2) d(2) a(2)]);hold on;
    plot([p(j,1)], [p(j,2)], '.', 'Color', 'r', 'MarkerSize', 10);hold on;
end
pitch = screwPitch/(2*pi);
theta0 = (4.5/pitch):0.01*pi:32*pi;
r1 = pitch * theta0;
% polarplot(theta0,r1,'b');
[x,y] = pol2cart(theta0,r1);
plot(x,y);hold on;
axis equal

```

附录 E 问题 4 Matlab 代码

blank4Angles.m:

```

% 根据两个把手的坐标 (xy)，获得四个角上点的坐标 (xy)
% 输入：两个把手的平面直角坐标,椅子总长度
% 输出：四个点，resp1 resp2为短边两点，resp2,resp3为长边两点
function [resp1,resp2,resp3,resp4] = blank4Angles(p0,p1,length)
    width = 0.3; % 板凳宽度
    k = (p0(2)-p1(2))/(p0(1)-p1(1));
    centerX = (p0(1)+p1(1))/2;
    centerY = (p0(2)+p1(2))/2;
    % 接下来求长边的中点坐标
    deltaX = length/2*(1/sqrt(1+k^2));
    deltaY = length/2*(k/sqrt(1+k^2));
    pm1 = [centerX+deltaX centerY+deltaY];
    pm2 = [centerX-deltaX centerY-deltaY];
    k2 = -1/k; % 中垂线
    deltaX = width/2*(1/sqrt(1+k2^2));
    deltaY = width/2*(k2/sqrt(1+k2^2));
    resp1 = [pm1(1)-deltaX, pm1(2)-deltaY];
    resp2 = [pm1(1)+deltaX, pm1(2)+deltaY];
    resp3 = [pm2(1)+deltaX, pm2(2)+deltaY];
    resp4 = [pm2(1)-deltaX, pm2(2)-deltaY];
end

```

getBenchDirection.m:

% 输入两个点的坐标，输出两个点向量与x轴正方向的夹角，即板凳方向

```
function theta = getBenchDirection(x0, y0, x1, y1)
```

```
    dx = x0 - x1;
    dy = y0 - y1;
    theta = atan2(dy, dx);
    if theta < 0
        theta = theta + 2*pi;
    end
```

```
end
```

getBodyLocation.m:

```
function [X, Y, THETA, STATE] = getBodyLocation(head_theta, state,
    screwPitch,Points,r,alpha,rad,rmin)
```

```
    sum = 224;
    l1 = 2.86;
    l2 = 1.65;
    X = zeros(1,sum);
    Y = zeros(1,sum);
    THETA = zeros(1,sum);
    STATE = zeros(1,sum);
    [x0,y0] = getXY(head_theta, state, screwPitch,Points,r,alpha);
    X(1) = x0;
    Y(1) = y0;
    THETA(1) = head_theta;
    STATE(1) = state;
    lastTheta = head_theta;
    lastArea = state;
    for i = 2:sum
        if i == 2
            l = l1;
        else
            l = l2;
        end
        [resTheta, resArea, x0, y0] = iterHandles4(lastTheta, lastArea, l, r, rad,
            Points,alpha,rmin);
%        disp(lastTheta)
        if x0 == 0
            [x,y] = getXY(resTheta, resArea, screwPitch,Points,r,alpha);
            X(i) = x;
            Y(i) = y;
        else
```

```

        X(i) = x0;
        Y(i) = y0;

    end

    THETA(i) = resTheta;
    STATE(i) = resArea;
    lastTheta = resTheta;
    lastArea = resArea;
end
end

```

getDirectionByTheta.m:

```

% 通过theta角计算当前速度方向与x轴正方向的夹角
% 参数:
% theta: 从0开始, 由内到外的角, 大于0
% state: 状态1是盘内, 2、3是圆上, 4是盘出
function resAngle = getDirectionByTheta(theta,state,alpha,rad)

    switch state
    case 1
        resAngle = atan(theta) + theta + pi;
    case 2
        resAngle = alpha - theta - pi/2;
    case 3
        resAngle = alpha + theta - pi/2 - rad;
    otherwise
        resAngle = atan(theta) + theta + pi;
    end
end
end

```

getSolution.m:

```

function [x1,y1,x2,y2,rad,r,rmin,alpha,beta] = getSolution()

    k = 1.7/2/pi;
    % 调头半径
    rmin = 4.5;

    % 盘入点的方向和坐标
    thetaIn = getDirectionByTheta(rmin/k,1);
    xIn = rmin*cos(rmin/k);
    yIn = rmin*sin(rmin/k);
    % 矩形长宽
    W = 2*rmin*cos(atan(thetaIn));
    L = 2*rmin*sin(atan(thetaIn));
    % 小圆半径和弧度角
    r = (W^2+L^2)/6/L;
    rad = pi-atan(W/(L-3*r));

```



```

% 大圆坐标
x1 = xIn + 2*r*cos(thetaIn-pi/2);
y1 = yIn + 2*r*sin(thetaIn-pi/2);
% 小圆坐标
x2 = -xIn - r*cos(thetaIn-pi/2);
y2 = -yIn - r*sin(thetaIn-pi/2);
% 圆心交线与x轴正方向的夹角
beta = atan2(y2-y1,x2-x1);
% 盘入螺线点的法向量与x轴正向夹角.
alpha = thetaIn+pi/2;
end

```

getThetaByT.m:

```

% 输入时间,龙头速度和螺距
% 时间零点为进入调头空间的时刻
function [result,state] = getThetaByT(tfinal,v,screwPitch,rad,r,rmin)
% 弧长
arc_length = [rad*r*2,rad*r];
spendT = arc_length/v;
k = screwPitch/(2*pi);
b = rmin/k;
start_t = (b*sqrt(b^2+1)+log(b+sqrt(b^2+1)))*k/2/v;
% 提高解的精度
options = odeset('RelTol',1e-10,'AbsTol',1e-10);
% 求偏微分方程的离散解
if tfinal<0
    [t, Xt] = ode45(@SunFun, [0 start_t-tfinal], 0, options);
    state = 1;
elseif tfinal<spendT(1) % 第一个圆弧上
    result = v*tfinal/(2*r);
    state = 2;
    return
elseif tfinal<sum(spendT) % 第二个圆弧上
    result = v*(tfinal-spendT(1))/r;
    state = 3;
    return
else % 盘出螺线
    [t, Xt] = ode45(@SunFun, [0 tfinal-sum(spendT)+start_t], 0, options);
    state = 4;
end
result = Xt(end);
% 微分方程函数, 状态导数
function xdot = SunFun(t, x)
    xdot = v/sqrt(x^2+1)/k;
end
end

```

getXY.m:

```
% 输入角度和状态, 螺距
function [x, y] = getXY(theta, state, screwPitch, Points, r, alpha)
    k = screwPitch/(2*pi);
    % 圆的圆心
    circle = [Points(1), Points(2); Points(3), Points(4)];
    switch state
        case 1
            r = k*theta;
            x = r*cos(theta);
            y = r*sin(theta);
        case 2
            x = circle(1,1) + 2*r*cos(alpha-theta);
            y = circle(1,2) + 2*r*sin(alpha-theta);
        case 3
            x = circle(2,1) + r*cos(alpha+theta);
            y = circle(2,2) + r*sin(alpha+theta);
        otherwise
            r = -k*theta;
            x = r*cos(theta);
            y = r*sin(theta);
    end
end
```

inspectionAngle.m:

```
screwPitch = 1.7;
head_v = 1;
uu = zeros(1,224);
vv = zeros(1,224);
t = 75;
% 获取两个圆的解析值
[x1,y1,x2,y2,rad,r,rmin,alpha0,beta0] = getSolution();
Points = [x1,y1,x2,y2];

% 画图
pitch = screwPitch/(2*pi);
theta0 = (4.5/pitch):0.01*pi:16*pi;
%   theta0 = 0:0.01*pi:16*pi;
r1 = pitch * theta0;
% polarplot(theta0,r1,'b');
[x,y] = pol2cart(theta0,r1);
plot(x,y);hold on;
[x,y] = pol2cart(theta0,-r1);
plot(x,y);hold on;
```

```

beta = 0:0.05*pi:2*pi;
x = cos(beta)*4.5;
y = sin(beta)*4.5;
plot(x, y);hold on;
x1=-0.760009116655535;
y1=-1.305726426346251;
x4=1.735932490181097;
y4=2.448401974553662;
r = 1.502708833894518;
alpha=16.571908201435512-5*pi;
plot([x1,x4],[y1,y4],'o');hold on;
beta = alpha:0.01*pi:(alpha+pi);
x = x1+cos(beta)*2*r;
y = y1+sin(beta)*2*r;
plot(x, y);hold on;
beta = (alpha-pi):0.01*pi:alpha;
x = x4+cos(beta)*r;
y = y4+sin(beta)*r;
plot(x, y);hold on;

[head_theta, state] = getThetaByT(t,head_v,screwPitch,rad,r,rmin);
[x0,y0] = getXY(head_theta, state, screwPitch, Points,r,alpha0);
% 获取整条队伍的信息
[X,Y,THETA,STATE] = getBodyLocation(head_theta, state, screwPitch,Points,r,alpha0,rad,rmin);

d = 0.1;
for i = 1:224
    angle = getDirectionByTheta(THETA(i),STATE(i),alpha0,rad);
    uu(i) = d*cos(angle);
    vv(i) = d*sin(angle);
end

% 画图
plot(X,Y,'.');hold on;
quiver(X,Y,uu,vv,0.7);
axis equal

```

iterHandles4.m:

```

function [resTheta,nextArea,resultX,resultY] =
    iterHandles4(theta,area,l,r,rad,Points,alpha,rmin)
resultX = 0;
resultY = 0;
k=1.7/2/pi;
Rmin=4.228886952526264;

```

```

% 圆的交点
XX = Points(1) + (Points(3)-Points(1))*2/3;
YY = Points(2) + (Points(4)-Points(2))*2/3;

if area == 1 % 当前在一区
    r0=k*theta;
%    disp(theta)
    dr0=2;
    f = @(x) (x^2+r0^2-1^2)-2*x*r0*cos((x-r0)/k);
    resR = fzero(f, [r0, r0+dr0]);
    resTheta = resR/k;
    nextArea=1;
elseif area == 2
    if theta<2*asin(1/r/4)
        [x0,y0] = getXY(theta,2,1.7,Points,r,alpha);
%        r0 = sqrt(x0^2+y0^2);
        dr0 = 3;
        f = @(x) (k*x*cos(x)-x0)^2 + (k*x*sin(x)-y0)^2 - 1^2;
        resTheta = fzero(f, [rmin/k, rmin/k+dr0]);
        nextArea=1;
    else
        resTheta = theta - 2*asin(1/r/4);
        nextArea=2;
    end
elseif area == 3
    if theta<2*asin(1/r/2)
        % 板凳在两圆弧之间
        [x0,y0] = getXY(theta, 3, 17,Points,r,alpha);

        % 已知参数
        c1 = [x0, y0]; % 第一个圆心坐标
        r1 = 1; % 第一个圆的半径
        c2 = [Points(1), Points(2)]; % 第二个圆心坐标
        r2 = 2*r; % 第二个圆的半径

        % 计算两个圆的交点
        d = norm(c2 - c1); % 两个圆心的距离
        a = (r1^2 - r2^2 + d^2)/(2*d);
        h = sqrt(r1^2 - a^2);
        p = c1 + a*(c2 - c1)/d;
        x1 = p(1) + h*(c2(2) - c1(2))/d;
        y1 = p(2) - h*(c2(1) - c1(1))/d;
        x2 = p(1) - h*(c2(2) - c1(2))/d;
        y2 = p(2) + h*(c2(1) - c1(1))/d;
    end
end

```

```

% 选择正确的解 (即在圆弧上的解)
if y1 < y2
    realX = x2;
    realY = y2;
else
    realX = x1;
    realY = y1;
end

d = sqrt((realX-Points(3))^2+(realY-Points(4))^2);
resTheta = rad - acos(((2*r)^2+(3*r)^2-d^2)/(2*2*r*3*r));
resultX = realX;
resultY = realY;

nextArea=2;
else
    resTheta = theta - 2*asin(1/r/2);
    nextArea=3;
end
elseif area == 4
    % 边界条件
    r0 = rmin;
    dr0 = 2;
    f = @(x) (x^2+r0^2-1^2)-2*x*r0*cos((x-r0)/k);
    Rmin2 = fzero(f,[r0,r0+2]);

    if l > sin(rad/2)*r*2 && theta*k < Rmin
        % 一个点在螺线, 另一个点在大圆
        % 圆心
        xr=Points(1);
        yr=Points(2);
        [x0,y0] = getXY(theta,4,1.7,Points,r,alpha);

        % 已知参数
        c1 = [x0, y0]; % 第一个圆心坐标
        r1 = 1;        % 第一个圆的半径
        c2 = [xr, yr]; % 第二个圆心坐标
        r2 = 2*r;      % 第二个圆的半径

        % 计算两个圆的交点
        d = norm(c2 - c1); % 两个圆心的距离
        a = (r1^2 - r2^2 + d^2)/(2*d);
        h = sqrt(r1^2 - a^2);
        p = c1 + a*(c2 - c1)/d;
        x1 = p(1) + h*(c2(2) - c1(2))/d;
        y1 = p(2) - h*(c2(1) - c1(1))/d;
    end
end
end

```

```

x2 = p(1) - h*(c2(2) - c1(2))/d;
y2 = p(2) + h*(c2(1) - c1(1))/d;

% 选择正确的解 (即在圆弧上的解)
if y1 < y2
    realX = x2;
    realY = y2;
else
    realX = x1;
    realY = y1;
end
d = sqrt((realX-Points(3))^2+(realY-Points(4))^2);
resTheta = rad - acos(((2*r)^2+(3*r)^2-d^2)/(2*2*r*3*r));
resultX = realX;
resultY = realY;

nextArea=2;
elseif theta*k<Rmin2
    % 一个在圆弧, 另一个在螺线
    % 圆心
    xr=Points(3);
    yr=Points(4);
    [x0,y0] = getXY(theta,4,1.7,Points,r,alpha);

    % 已知参数
    c1 = [x0, y0]; % 第一个圆心坐标
    r1 = 1;        % 第一个圆的半径
    c2 = [xr, yr]; % 第二个圆心坐标
    r2 = r;        % 第二个圆的半径

    % 计算两个圆的交点
    d = norm(c2 - c1); % 两个圆心的距离
    a = (r1^2 - r2^2 + d^2)/(2*d);
    h = sqrt(r1^2 - a^2);
    p = c1 + a*(c2 - c1)/d;
    x1 = p(1) + h*(c2(2) - c1(2))/d;
    y1 = p(2) - h*(c2(1) - c1(1))/d;
    x2 = p(1) - h*(c2(2) - c1(2))/d;
    y2 = p(2) + h*(c2(1) - c1(1))/d;

    % 选择正确的解 (即在圆弧上的解)
    if x1 < x2
        realX = x2;
        realY = y2;
    else
        realX = x1;
        realY = y1;
    end
end

```

```

end

d = sqrt((realX-XX)^2+(realY-YY)^2);
resTheta = 2*asin(d/2/r);
resultX = realX;
resultY = realY;

nextArea=3;
else
    % 都在螺线上
    r0=k*theta;
    dr0=2;
    f = @(x) (x^2+r0^2-1^2)-2*x*r0*cos((x-r0)/k);
    resR = fzero(f,[r0-dr0,r0]);
    resTheta = resR/k;
    nextArea=4;
end
end
end
end

```

problem4.m:

```

screwPitch = 1.7;
head_v = 1;
XY = zeros(448, 201);
VV = zeros(224, 201);

% 获取两个圆的解析值
[x1,y1,x2,y2,rad,r,rmin,alpha0,beta0] = getSolution();
Points = [x1,y1,x2,y2];

for t = -100:100

    [head_theta, state] = getThetaByT(t,head_v,screwPitch,rad,r,rmin);
    [x0,y0] = getXY(head_theta, state, screwPitch, Points,r,alpha0);
    % 获取整条队伍的信息
    [X,Y,THETA,STATE] = getBodyLocation(head_theta, state, screwPitch,Points,r,alpha0,rad,rmin);

    % 获取速度
    lastTheta = THETA(1);
    lastState = STATE(1);
    lastX = X(1);
    lastY = Y(1);
    lastV = head_v;
    V = zeros(1,224);

```

```

V(1) = lastV;

for i = 2:length(THETA)

%       % 画板凳
%       if i == 2
%           len = 2.86;
%       else
%           len = 1.65;
%       end
%       [p1,p2,p3,p4] = blank4Angles([lastX,lastY],[X(i),Y(i)],len);
%       plot([p1(1),p2(1),p3(1),p4(1),p1(1)],[p1(2),p2(2),p3(2),p4(2),p1(2)]);hold on;

    theta = THETA(i);
    alpha = getDirectionByTheta(lastTheta,lastState,alpha0,rad);
    beta = getDirectionByTheta(theta,STATE(i),alpha0,rad);
    omega = getBenchDirection(lastX, lastY, X(i), Y(i));
    v = abs(lastV*cos(omega-alpha)/cos(beta-omega));
    V(i) = v;
    lastV = v;
    lastTheta = THETA(i);
    lastState = STATE(i);
    lastX = X(i);
    lastY = Y(i);

end

% 存入xy坐标数据
for i = 1:224
    XY(2*i-1,t+101) = X(i);
    XY(2*i,t+101) = Y(i);
end
VV(:,t+101) = V';

% 画图
clf;
pitch = screwPitch/(2*pi);
theta0 = (rmin/pitch):0.01*pi:16*pi;
% theta0 = 0:0.01*pi:16*pi;
r1 = pitch * theta0;
% polarplot(theta0,r1,'b');
[x,y] = pol2cart(theta0,r1);
plot(x,y);hold on;
[x,y] = pol2cart(theta0,-r1);
plot(x,y);hold on;

```



```

% 调头空间
beta = 0:0.05*pi:2*pi;
x = cos(beta)*4.5;
y = sin(beta)*4.5;
fill(x, y, 'yellow', 'FaceAlpha', 0.3);hold on;
% 转弯圆心
plot([x1,x2],[y1,y2], 'o');hold on;
% 圆弧
beta = (alpha0-6*pi):-0.01*pi:beta0;
x = x1+cos(beta)*2*r;
y = y1+sin(beta)*2*r;
plot(x, y, 'r', 'Linewidth', 1);hold on;
beta = beta0:-0.01*pi:(alpha0-8*pi);
x = x2+cos(beta)*r;
y = y2+sin(beta)*r;
plot(x, y, 'r', 'Linewidth', 1);hold on;

% % 圆相接点
% plot([1.3662],[-0.2941], 'o');hold on
% % 龙头圆
% omega = 0:0.01:2*pi;
% x = X(1) + 2.86*cos(omega);
% y = Y(1) + 2.86*sin(omega);
% plot(x,y);

plot(X,Y, '.');hold on;
plot(X,Y, 'b', 'Linewidth', 1);
axis equal
pause(0.01)

end

% % 检验
% figure
% plot(1:224,VV(:,175));

```

problem4Plus.m:

```

maxL = 2.86;
% 求刚好不卡死下的外接圆半径
fun = @(x) getChordLength(x)-maxL;
rmin = fzero(fun,[4.0,4.5])

% 求弦长
function result = getChordLength(rmin)
    k = 1.7/2/pi;

```

```

thetaIn = getDirectionByTheta(rmin/k,1);
% 矩形长宽
W = 2*rmin*cos(atan(thetaIn));
L = 2*rmin*sin(atan(thetaIn));
% 小圆半径和弧度角
r = (W^2+L^2)/6/L;
rad = pi-atan(W/(L-3*r));
result = r*sin(rad/2)*2;
end

```

附录 F 问题 5 Matlab 代码

problem5.m:

```

screwPitch = 1.7;
head_v = 1;
startT=0;
step=0.05;
multi=1000;
XY = zeros(448, multi+1);
VV = zeros(224, multi+1);

% 获取两个圆的解析值
[x1,y1,x2,y2,rad,r,rmin,alpha0,beta0] = getSolution();
Points = [x1,y1,x2,y2];

for head_v=1.221:0.001:1.231
    disp(head_v)
    for i1 = 0:multi
        t=-startT+i1*step;
        [head_theta, state] = getThetaByT(t,head_v,screwPitch,rad,r,rmin);
        [x0,y0] = getXY(head_theta, state, screwPitch, Points,r,alpha0);
        % 获取整条队伍的信息
        [X,Y,THETA,STATE] = getBodyLocation(head_theta, state,
            screwPitch,Points,r,alpha0,rad,rmin);

        % 获取速度
        lastTheta = THETA(1);
        lastState = STATE(1);
        lastX = X(1);
        lastY = Y(1);
        lastV = head_v;
        V = zeros(1,224);
        V(1) = lastV;
    end
end

```

```

for i = 2:length(THETA)
    theta = THETA(i);
    alpha = getDirectionByTheta(lastTheta,lastState,alpha0,rad);
    beta = getDirectionByTheta(theta,STATE(i),alpha0,rad);
    omega = getBenchDirection(lastX, lastY, X(i), Y(i));
    v = abs(lastV*cos(omega-alpha)/cos(beta-omega));
    V(i) = v;
    lastV = v;
    lastTheta = THETA(i);
    lastState = STATE(i);
    lastX = X(i);
    lastY = Y(i);
end
VV(:,i1+1) = V';

end
maxV=max(VV);
mV=max(maxV);
if mV>2
    plot(-startT:step:-startT+step*multi,maxV);
    break;
end
%plot(-startT:step:-startT+step*multi,maxV,'LineWidth',2);
end

```